

第10章 AI工程架构与用户反馈

到目前为止，本书已经涵盖了各种适配基础模型到特定应用的技术。本章将讨论如何将这些技术整合起来，构建成功的产品。

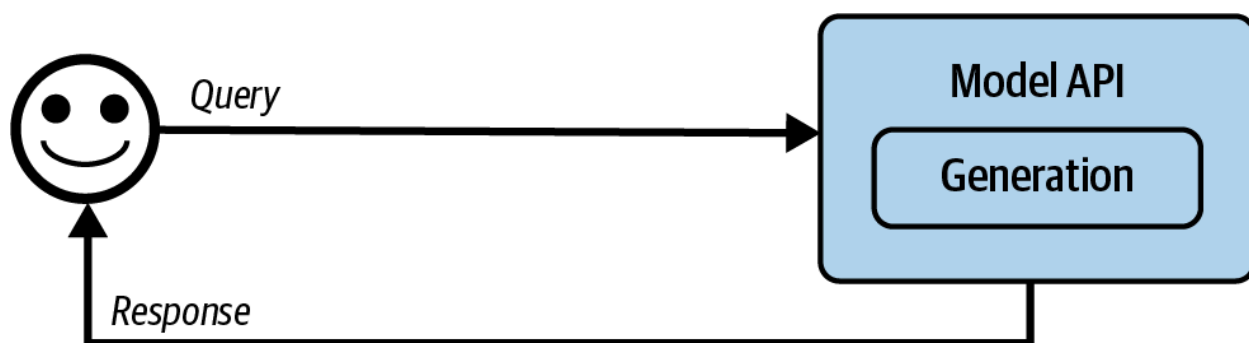
鉴于可用的AI工程技术和工具范围广泛，选择正确的工具可能令人感到压力。为简化这一过程，本章采用渐进式方法。它从AI应用的最简架构开始，强调该架构的挑战，然后逐步添加组件来解决这些问题。

我们可以无休止地推论如何构建成功的应用，但确定一个应用是否真正实现其目标的唯一方法是将其呈现给用户。用户反馈一直是指导产品开发的宝贵资源，但对于AI应用，用户反馈作为改进模型的数据源有着更为关键的作用。对话界面使用户更容易提供反馈，但也使开发者更难提取信号。本章将讨论不同类型的对话式AI反馈，以及如何设计一个系统来收集正确的反馈而不影响用户体验。

AI工程架构

一个完整的AI架构可能很复杂。本节遵循一个团队在生产中可能遵循的过程，从最简单的架构开始，逐步添加更多组件。尽管AI应用多种多样，但它们共享许多共同组件。这里提出的架构已在多家公司得到验证，适用于广泛的应用，但某些应用可能有所不同。

在最简单的形式中，你的应用接收查询并将其发送给模型。模型生成响应，然后返回给用户，如图10-1所示。这里没有上下文增强，没有护栏，也没有优化。模型API框指的是第三方API（如OpenAI、Google、Anthropic）和自托管模型。第9章讨论了为自托管模型构建推理服务器。



<图10-1的描述：运行AI应用的最简架构>

从这个简单的架构开始，你可以根据需要添加更多组件。这个过程可能如下所示：

1. 通过给模型提供外部数据源和信息收集工具，增强输入模型的上下文。
2. 设置护栏以保护你的系统和用户。
3. 添加模型路由器和网关，以支持复杂的管道并增加更多安全性。
4. 通过缓存优化延迟和成本。
5. 添加复杂逻辑和写入操作，以最大化系统能力。

本章遵循我在生产中常见的进程。然而，每个人的需求都不同。你应该遵循对你的应用最有意义的顺序。

监控和可观测性对于任何应用的质量控制和性能改进都至关重要，将在这个过程结束时讨论。编排，即将所有这些组件链接在一起，将在之后讨论。

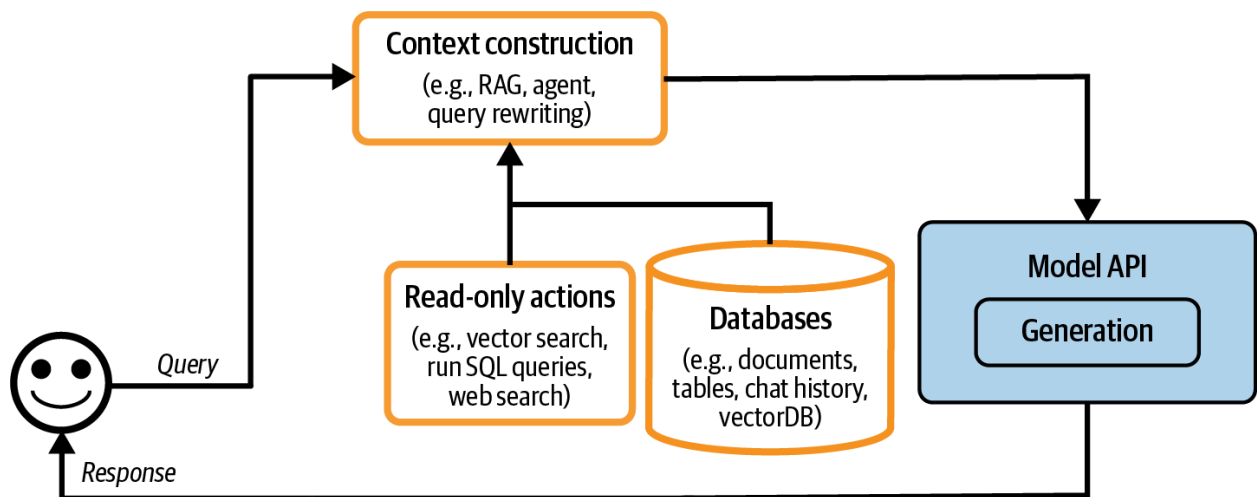
步骤1. 增强上下文

平台的初始扩展通常涉及添加机制，使系统能够构建模型回答每个查询所需的相关上下文。如第6章所讨论的，上下文可以通过各种检索机制构建，包括文本检索、图像检索和表格数据检索。上下文还可以使用工具增强，这些工具允许模型通过API(如网络搜索、新闻、天气、事件等)自动收集信息。

上下文构建就像基础模型的特征工程。它为模型提供生成输出所需的必要信息。由于其在系统输出质量中的核心作用，上下文构建几乎得到了所有模型API提供商的普遍支持。例如，像OpenAI、Claude和Gemini这样的提供商允许用户上传文件，并允许他们的模型使用工具。

然而，正如模型在能力上有所不同，这些提供商在上下文构建支持上也有所不同。例如，它们可能对你可以上传的文档类型和数量有限制。专业的RAG解决方案可能让你上传尽可能多的文档，只要你的向量数据库能容纳，但通用模型API可能只允许你上传少量文档。不同的框架在检索算法和其他检索配置(如块大小)上也有所不同。同样，对于工具使用，解决方案在支持的工具类型和执行模式上也有所不同，例如它们是否支持并行函数执行或长时间运行的任务。

添加了上下文构建后，架构现在如图10-2所示。



<图10-2的描述:带有上下文构建的平台架构>

步骤2. 设置护栏

护栏有助于减轻风险并保护你和用户。它们应该放置在任何存在风险暴露的地方。一般来说, 它们可以分为输入和输出周围的护栏。

输入护栏

输入护栏通常防范两类风险: 向外部API泄露私人信息和执行可能危害系统的恶意提示。第5章讨论了攻击者通过提示黑客攻击利用应用的多种方式, 以及如何防御这些攻击。虽然你可以减轻风险, 但由于模型生成响应的固有性质以及不可避免的人为失误, 风险永远无法完全消除。

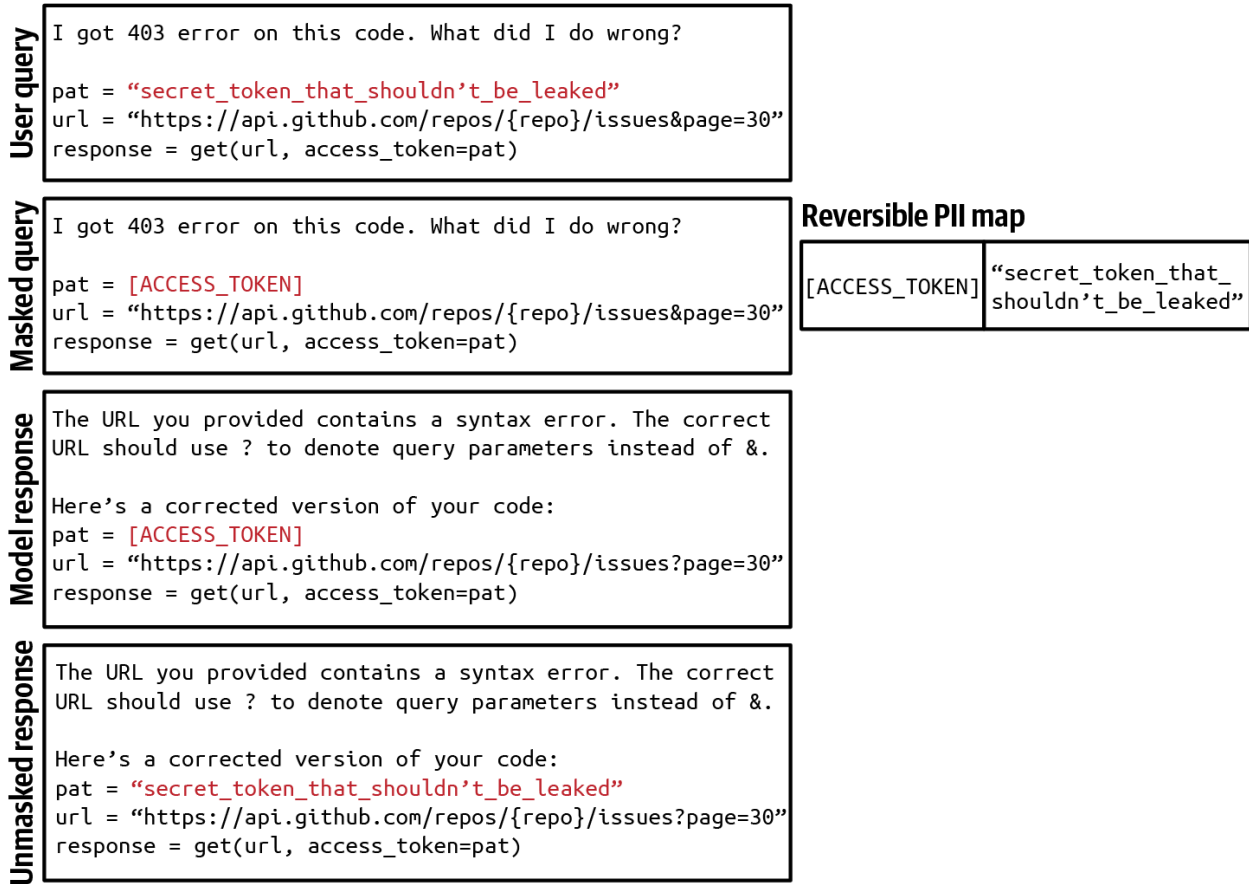
向外部API泄露私人信息是使用外部模型API时的特定风险, 当你需要将数据发送到组织外部时。这可能出于多种原因发生, 包括以下几点:

- 员工将公司机密或用户私人信息复制到提示中, 并将其发送给第三方API。
- 应用开发者将公司内部政策和数据放入应用的系统提示中。
- 工具从内部数据库检索私人信息并将其添加到上下文中。

在使用第三方API时, 没有万无一失的方法来消除潜在的泄漏。然而, 你可以通过护栏来减轻它们。你可以使用许多可用的工具自动检测敏感数据。要检测的敏感数据由你指定。常见的敏感数据类别包括:

- 个人信息(身份证号码、电话号码、银行账户)
- 人脸
- 与公司知识产权或特权信息相关的特定关键词和短语

许多敏感数据检测工具使用AI来识别潜在的敏感信息，例如确定字符串是否类似有效的家庭地址。如果发现查询包含敏感信息，你有两个选择：阻止整个查询或从中删除敏感信息。例如，你可以用占位符[电话号码]掩盖用户的电话号码。如果生成的响应包含此占位符，请使用将该占位符映射到原始信息的PII反向字典，以便可以取消掩盖，如图10-3所示。



<图10-3的描述：使用反向PII映射掩盖和取消掩盖PII信息，以避免将其发送到外部API的示例>

输出护栏

模型可能以多种方式失败。输出护栏有两个主要功能：

- 1. 捕获输出失败
- 2. 指定处理不同失败模式的策略

要捕获不符合标准的输出，你需要了解失败是什么样子的。最容易检测的失败是当模型在不应该的情况下返回空响应。对于不同的应用，失败看起来不同。以下是两个主要类别中的一些常见失败：质量和安全性。质量失败在第4章中讨论，安全性失败在第5章中讨论。我快速提及一些失败作为回顾：

质量

- 格式错误的响应, 不符合预期的输出格式。例如, 应用期望JSON, 而模型生成无效JSON。
- 模型产生的事实不一致的响应。
- 普遍不好的响应。例如, 你要求模型写一篇文章, 但那篇文章质量很差。

安全性

- 包含种族主义内容、性内容或非法活动的有毒响应。
- 包含私人和敏感信息的响应。
- 触发远程工具和代码执行的响应。
- 错误描述你的公司或竞争对手的品牌风险响应。

回顾第5章, 对于安全措施, 重要的是不仅要跟踪安全失败, 还要跟踪错误拒绝率。可能存在过于安全的系统, 例如阻止甚至合法请求的系统, 这会中断用户工作负载并导致用户不满。

许多失败可以通过简单的重试逻辑来缓解。AI模型具有概率性, 这意味着如果你再次尝试查询, 可能会得到不同的响应。例如, 如果响应为空, 尝试X次或直到获得非空响应。同样, 如果响应格式错误, 尝试直到响应格式正确。

然而, 这种重试策略可能会增加延迟和成本。每次重试意味着另一轮API调用。如果在失败后进行重试, 用户感知的延迟将加倍。为了减少延迟, 你可以并行进行调用。例如, 对于每个查询, 不是等待第一个查询失败后再重试, 而是同时向模型发送两次这个查询, 获得两个响应, 然后选择更好的一个。这增加了冗余API调用的数量, 同时保持延迟可管理。

对于棘手的请求, 人工干预也很常见。例如, 你可以将包含特定短语的查询转给人工操作员。一些团队使用专门的模型来决定何时将对话转给人工。例如, 一个团队在情感分析模型检测到用户消息中的愤怒时, 将对话转给人工操作员。另一个团队在一定次数的对话轮次后转为人工, 以防止用户陷入循环。

护栏实施

护栏带来权衡。一种是可靠性与延迟的权衡。虽然承认护栏的重要性, 但一些团队告诉我延迟更重要。这些团队决定不实施护栏, 因为它们可能显著增加应用的延迟。

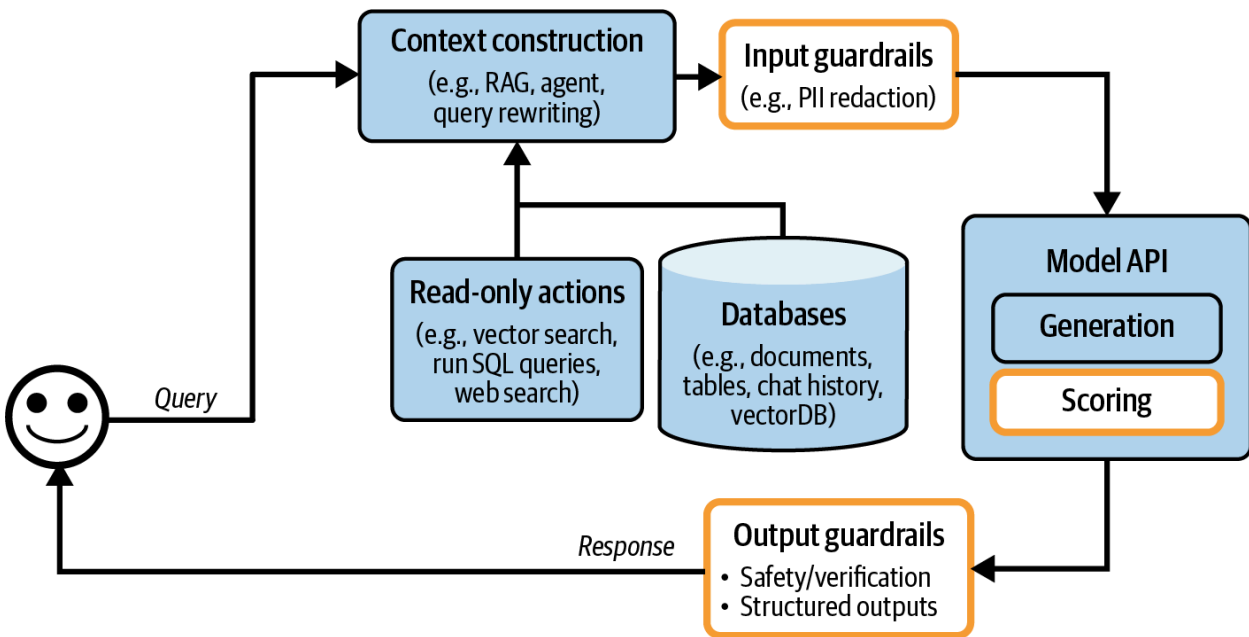
输出护栏在流式完成模式下可能效果不佳。默认情况下, 整个响应在显示给用户之前生成, 这可能需要很长时间。在流式完成模式下, 新的标记在生成时就会流向用户, 减少用户等待看到响应的的时间。缺点是很难评估部分响应, 因此在系统护栏确定它们应该被阻止之前, 不安全的响应可能会流向用户。

你需要实施多少护栏也取决于你是自托管模型还是使用第三方API。虽然你可以在两者之上实施护栏, 但第三方API可以减少你需要实施的护栏, 因为API提供商通常为你提供许多开箱即用的护栏。同时, 自托管意味着你不需要向外部发送请求, 这减少了对许多类型的输入护栏的需求。

鉴于应用可能失败的许多不同地方，护栏可以在多个层次实施。模型提供商为其模型提供护栏，以使其模型更好、更安全。然而，模型提供商必须平衡安全性和灵活性。限制可能使模型更安全，但也可能使其对特定用例的可用性降低。

护栏也可以由应用开发者实施。许多技术在"防御提示攻击"中讨论。可以开箱即用的护栏解决方案包括Meta的Purple Llama、NVIDIA的NeMo Guardrails、Azure的PyRIT、Azure的AI内容过滤器、Perspective API和OpenAI的内容审核API。由于输入和输出风险的重叠，护栏解决方案可能为输入和输出提供保护。一些模型网关也提供护栏功能，将在下一节讨论。

添加了护栏后，架构如图10-4所示。我将评分器放在模型API下，因为评分器通常由AI提供支持，即使评分器通常比生成模型更小更快。然而，评分器也可以放在输出护栏框中。



<图10-4的描述: 添加了输入和输出护栏的应用架构>

步骤3. 添加模型路由器和网关

随着应用增长到涉及更多模型，路由器和网关出现，帮助你管理服务多个模型的复杂性和成本。

路由器

你可以为不同类型的查询使用不同的解决方案，而不是对所有查询使用一个模型。这种方法有几个好处。首先，它允许专门化模型，这些模型可能在特定查询上比通用模型表现更好。例如，你可以有一个专门用于技术故障排除的模型，另一个专门用于账单。其次，这可以帮助你节省成本。与使用一个昂贵的模型处理所有查询相比，你可以将更简单的查询路由到更便宜的模型。

路由器通常由意图分类器组成，预测用户试图做什么。根据预测的意图，查询被路由到适当的解决方案。例如，考虑与客户支持聊天机器人相关的不同意图：

- 如果用户想要重置密码，将其路由到关于密码恢复的FAQ页面。
- 如果请求是纠正账单错误，将其路由给人工操作员。
- 如果请求是关于排除技术问题，将其路由到专门排除故障的聊天机器人。

意图分类器可以防止系统参与超出范围的对话。如果查询被认为不适当，聊天机器人可以礼貌地拒绝响应，使用一种预设回复，而不浪费API调用。例如，如果用户询问你会在即将到来的选举中投票给谁，聊天机器人可以回应："作为聊天机器人，我没有投票的能力。如果你有关于我们产品的问题，我很乐意帮忙。"

意图分类器可以帮助系统检测模糊查询并要求澄清。例如，对于查询"冻结"，系统可能会问："你想冻结你的账户还是在谈论天气？"或者简单地问："对不起，你能详细说明吗？"

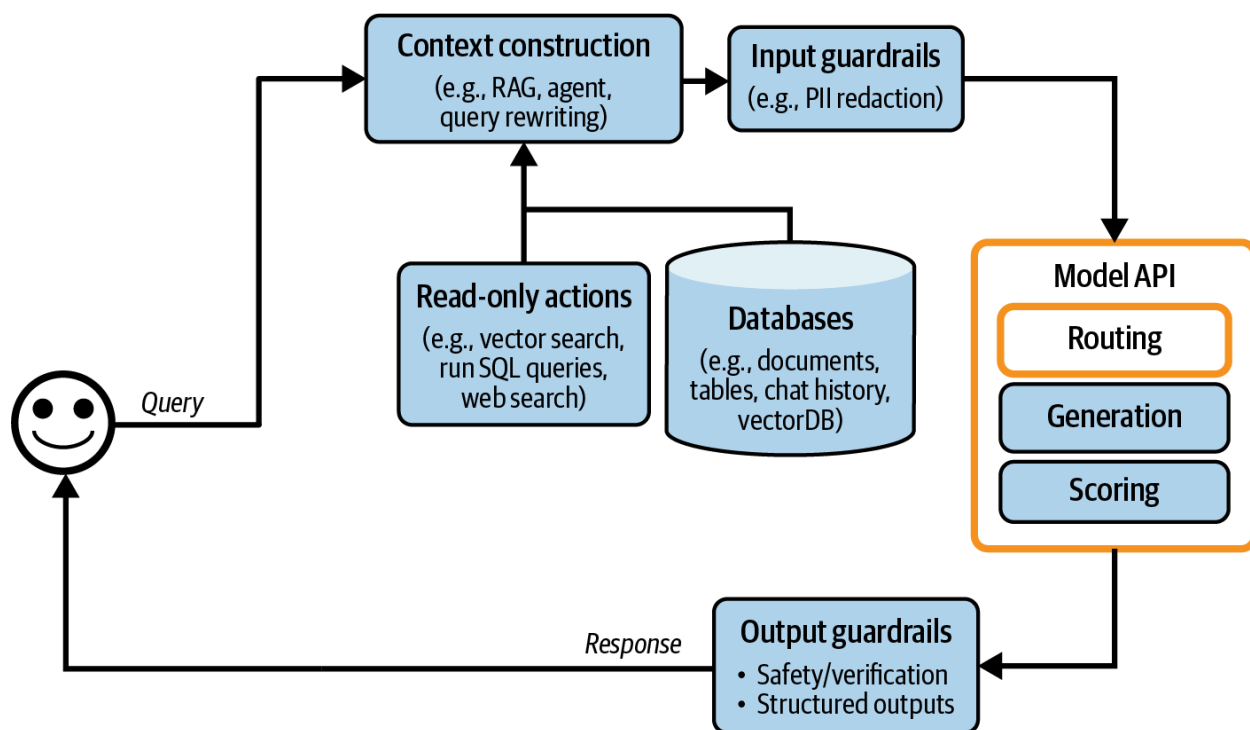
其他路由器可以帮助模型决定下一步做什么。例如，对于一个能够执行多种操作的代理，路由器可以采取下一个操作预测器的形式：模型应该使用代码解释器还是搜索API？对于具有记忆系统的模型，路由器可以预测模型应该从记忆层次结构的哪一部分提取信息。想象一个用户在当前对话中附加了一个提到墨尔本的文档。后来，用户问："墨尔本最可爱的动物是什么？"模型需要决定是依赖附加文档中的信息还是为此查询搜索互联网。

意图分类器和下一个操作预测器可以在基础模型上实现。许多团队将较小的语言模型如GPT-2、BERT和Llama 7B作为他们的意图分类器。许多团队选择从头训练更小的分类器。路由器应该快速且便宜，以便可以使用多个而不会产生显著的额外延迟和成本。

当将查询路由到具有不同上下文限制的模型时，可能需要相应地调整查询的上下文。考虑一个计划用于具有4K上下文限制的模型的1,000词元查询。然后系统执行一个操作，例如网络搜索，带回8,000词元上下文。你可以截断查询的上下文以适应原定的模型，或者将查询路由到具有更大上下文限制的模型。

因为路由通常由模型完成，我在图10-5的模型API框中放入了路由。与评分器一样，路由器通常比用于生成的模型更小。

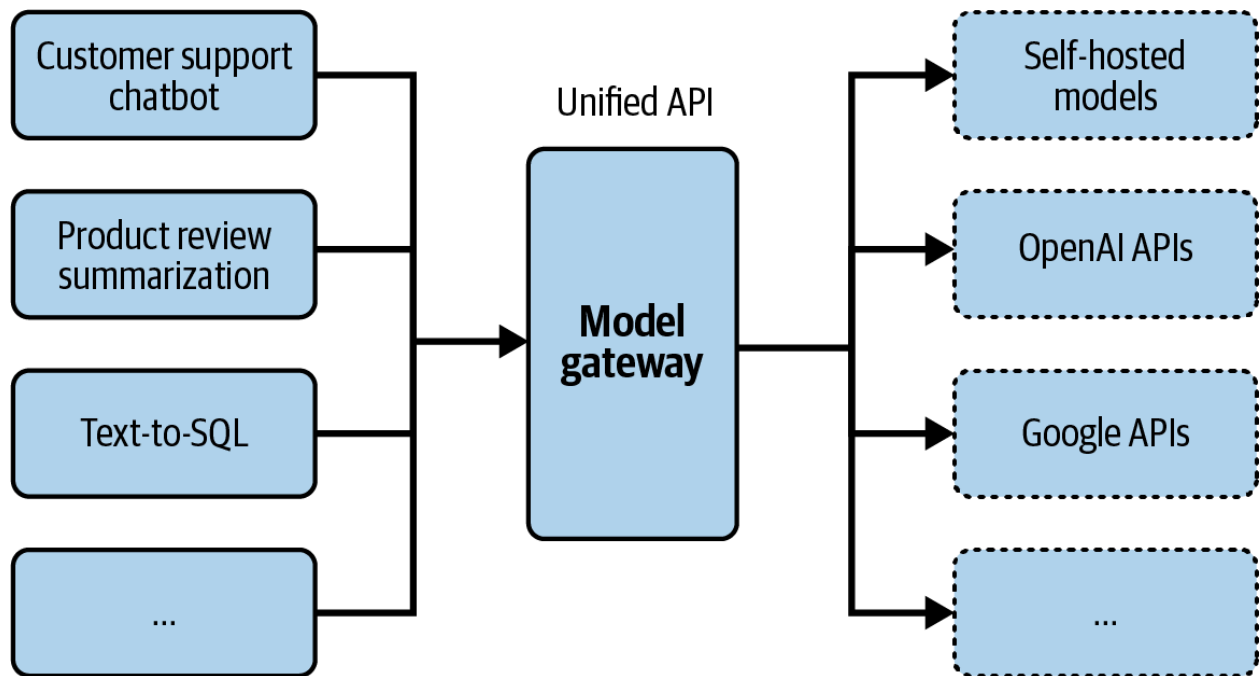
将路由器与其他模型分组在一起使模型更容易管理。然而，重要的是要注意，路由通常在检索之前发生。例如，在检索之前，路由器可以帮助确定查询是否在范围内，如果是，它是否需要检索。路由也可以在检索之后发生，例如确定查询是否应该路由给人工操作员。然而，路由 - 检索 - 生成 - 评分是一个更常见的AI应用模式。



<图10-5的描述:路由帮助系统为每个查询使用最佳解决方案>

网关

模型网关是一个中间层，允许你的组织以统一和安全的方式与不同的模型接口。模型网关最基本的功能是为不同的模型提供统一的接口，包括自托管模型和商业API背后的模型。模型网关使维护代码更容易。如果模型API发生变化，你只需更新网关，而不需要更新所有依赖此API的应用。图10-6显示了模型网关的高级可视化。



<图10-6的描述：模型网关提供统一接口来使用不同模型>

在最简单的形式中，模型网关是一个统一的包装器。以下代码示例展示了模型网关的实现方式。它不是功能性的，因为它不包含任何错误检查或优化：

```

import google.generativeai as genai
import openai

def openai_model(input_data, model_name, max_tokens):
    openai.api_key = os.environ["OPENAI_API_KEY"]
    response = openai.Completion.create(
        engine=model_name,
        prompt=input_data,
        max_tokens=max_tokens
    )
    return {"response": response.choices[0].text.strip()}

def gemini_model(input_data, model_name, max_tokens):
    genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
    model = genai.GenerativeModel(model_name=model_name)
    response = model.generate_content(input_data, max_tokens=max_tokens)
    return {"response": response["choices"][0]["message"]["content"]}

@app.route('/model', methods=['POST'])
def model_gateway():

```

```
data = request.get_json()
model_type = data.get("model_type")
model_name = data.get("model_name")
input_data = data.get("input_data")
max_tokens = data.get("max_tokens")

if model_type == "openai":
    result = openai_model(input_data, model_name, max_tokens)
elif model_type == "gemini":
    result = gemini_model(input_data, model_name, max_tokens)
return jsonify(result)
```

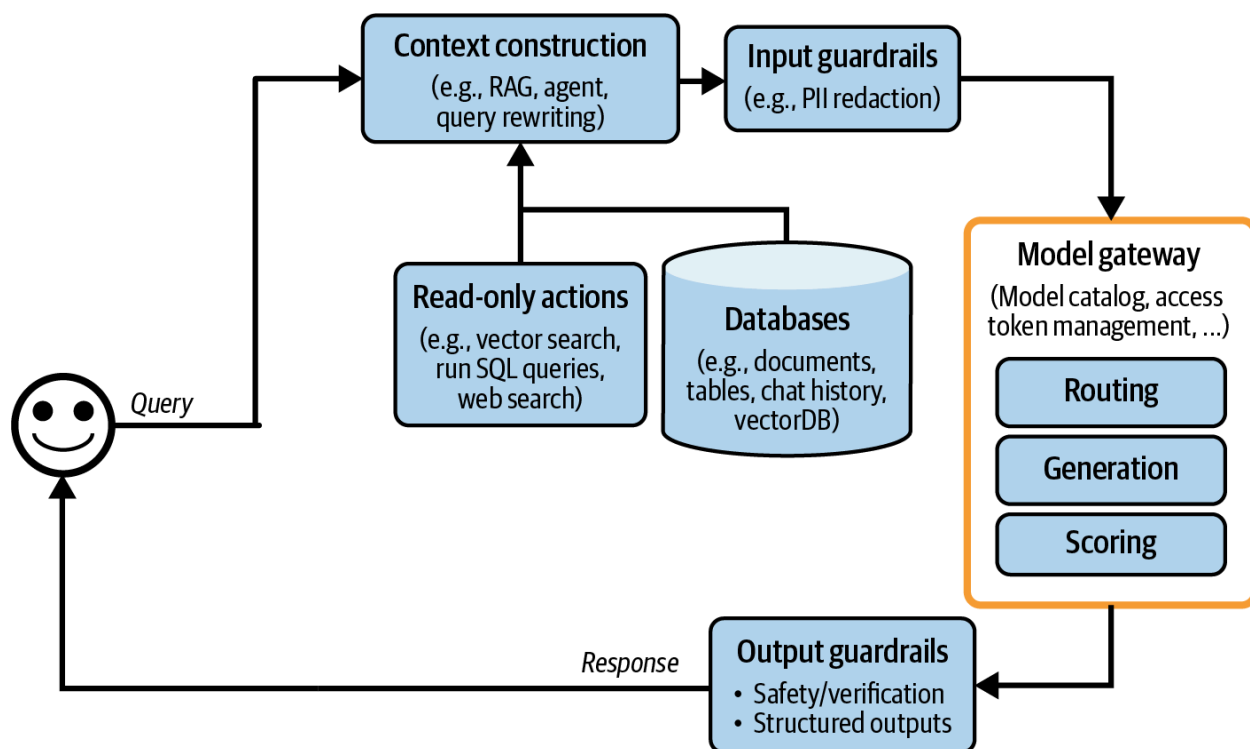
模型网关提供访问控制和成本管理。你不是给每个想要访问OpenAI API的人你的组织令牌(这很容易泄露),而是只给人们访问模型网关的权限,创建一个集中和受控的访问点。网关还可以实施细粒度访问控制,指定哪个用户或应用应该有权访问哪个模型。此外,网关可以监控和限制API调用的使用,防止滥用并有效管理成本。

模型网关也可用于实施故障转移策略,以克服速率限制或API故障(后者不幸很常见)。当主要API不可用时,网关可以将请求路由到替代模型,在短暂等待后重试,或以其他方式优雅地处理故障。这确保你的应用可以平稳运行,不会中断。

由于请求和响应已经流经网关,它是实施其他功能的好地方,如负载均衡、日志记录和分析。一些网关甚至提供缓存和护栏。

由于网关相对容易实施,有许多现成的网关。例如包括Portkey的AI Gateway、MLflow AI Gateway、Wealthsimple的LLM Gateway、TrueFoundry、Kong和Cloudflare。

在我们的架构中,网关现在替代了模型API框,如图10-7所示。



<图10-7的描述: 添加了路由和网关模块的架构>

注意

类似的抽象层，如工具网关，对于访问各种工具也很有用。由于截至撰写本文时这不是一种常见模式，本书不会讨论它。

步骤4. 通过缓存减少延迟

缓存长期以来一直是软件应用的组成部分，用于减少延迟和成本。软件缓存的许多想法可用于AI应用。推理缓存技术，包括KV缓存和提示缓存，在第9章中讨论。本节重点关注系统缓存。由于缓存是一项有大量现有文献的古老技术，本书将只大致介绍它。一般来说，有两种主要的系统缓存机制：精确缓存和语义缓存。

精确缓存

使用精确缓存，缓存项仅在请求这些精确项目时使用。例如，如果用户要求模型总结一个产品，系统会检查缓存，看是否存在此确切产品的摘要。如果存在，获取此摘要。如果不存在，总结产品并缓存摘要。

精确缓存也用于基于嵌入的检索，以避免冗余的向量搜索。如果传入的查询已经在向量搜索缓存中，获取缓存的结果。如果不在，为此查询执行向量搜索并缓存结果。

对于涉及多个步骤(例如, 思维链) 和/或耗时操作(例如, 检索、SQL执行或网络搜索)的查询, 缓存尤其有吸引力。

精确缓存可以使用内存存储实现, 以便快速检索。然而, 由于内存存储有限, 缓存也可以使用像 PostgreSQL、Redis或分层存储的数据库实现, 以平衡速度和存储容量。有一个驱逐策略对于管理缓存大小和维护性能至关重要。常见的驱逐策略包括最近最少使用(LRU)、最不经常使用(LFU)和先进先出(FIFO)。

将查询保留在缓存中的时间长短取决于该查询再次被调用的可能性。用户特定的查询, 如"我最近订单的状态是什么?", 不太可能被其他用户重用, 因此不应该被缓存。同样, 缓存时间敏感的查询如"天气如何?"也没有太大意义。许多团队训练一个分类器来预测查询是否应该被缓存。

警告

如果处理不当, 缓存可能导致数据泄漏。想象你为一家电子商务网站工作, 用户X提出了一个看似通用的问题:"电子产品的退货政策是什么?"因为你的退货政策取决于用户的会员资格, 系统首先检索用户X的信息, 然后生成包含X信息的响应。误将此查询视为通用问题, 系统缓存了答案。后来, 当用户Y提出相同问题时, 返回了缓存的结果, 向Y透露了X的信息。

语义缓存

与精确缓存不同, 即使缓存项仅在语义上相似而非完全相同于传入的查询, 也会使用它们。想象一个用户问:"越南的首都是什么?"模型回答:"河内"。后来, 另一个用户问:"越南的首都城市是什么?", 这在语义上是相同的问题, 但措辞略有不同。通过语义缓存, 系统可以重用第一个查询的答案, 而不是从头计算新查询。重用相似查询增加了缓存的命中率, 并可能降低成本。然而, 语义缓存可能降低模型的性能。

语义缓存仅在你有可靠的方法确定两个查询是否相似时有效。一种常见的方法是使用语义相似性, 如第3章所讨论的。作为复习, 语义相似性的工作原理如下:

1. 对于每个查询, 使用嵌入模型生成其嵌入。
2. 使用向量搜索查找与当前查询嵌入最高相似度分数的缓存嵌入。假设这个相似度分数是X。
3. 如果X高于某个相似度阈值, 则认为缓存的查询是相似的, 并返回缓存的结果。如果不是, 处理当前查询并将其与其嵌入和结果一起缓存。

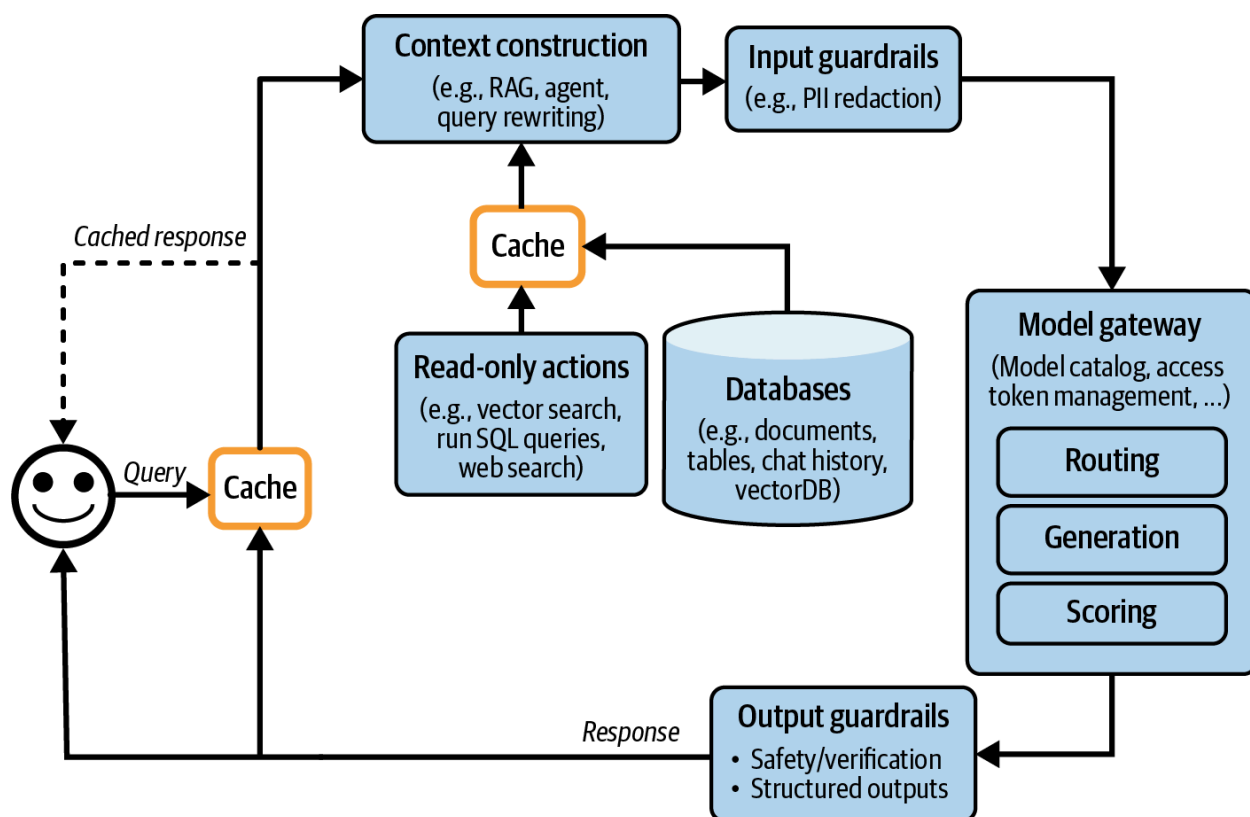
这种方法需要一个向量数据库来存储缓存查询的嵌入。

与其他缓存技术相比, 语义缓存的价值更加可疑, 因为它的许多组件容易失效。其成功依赖于高质量的嵌入、功能齐全的向量搜索和可靠的相似性度量。设置正确的相似度阈值也可能很棘手, 需要大量尝试和错误。如果系统错误地将传入查询误认为与另一个查询类似, 从缓存中获取的返回响应将是不正确的。

此外, 语义缓存可能既耗时又计算密集, 因为它涉及向量搜索。此向量搜索的速度和成本取决于缓存嵌入的大小。

如果缓存命中率高，即很大一部分查询可以有效地通过利用缓存结果来回答，语义缓存可能仍然值得。然而，在纳入语义缓存的复杂性之前，请确保评估相关的效率、成本和性能风险。

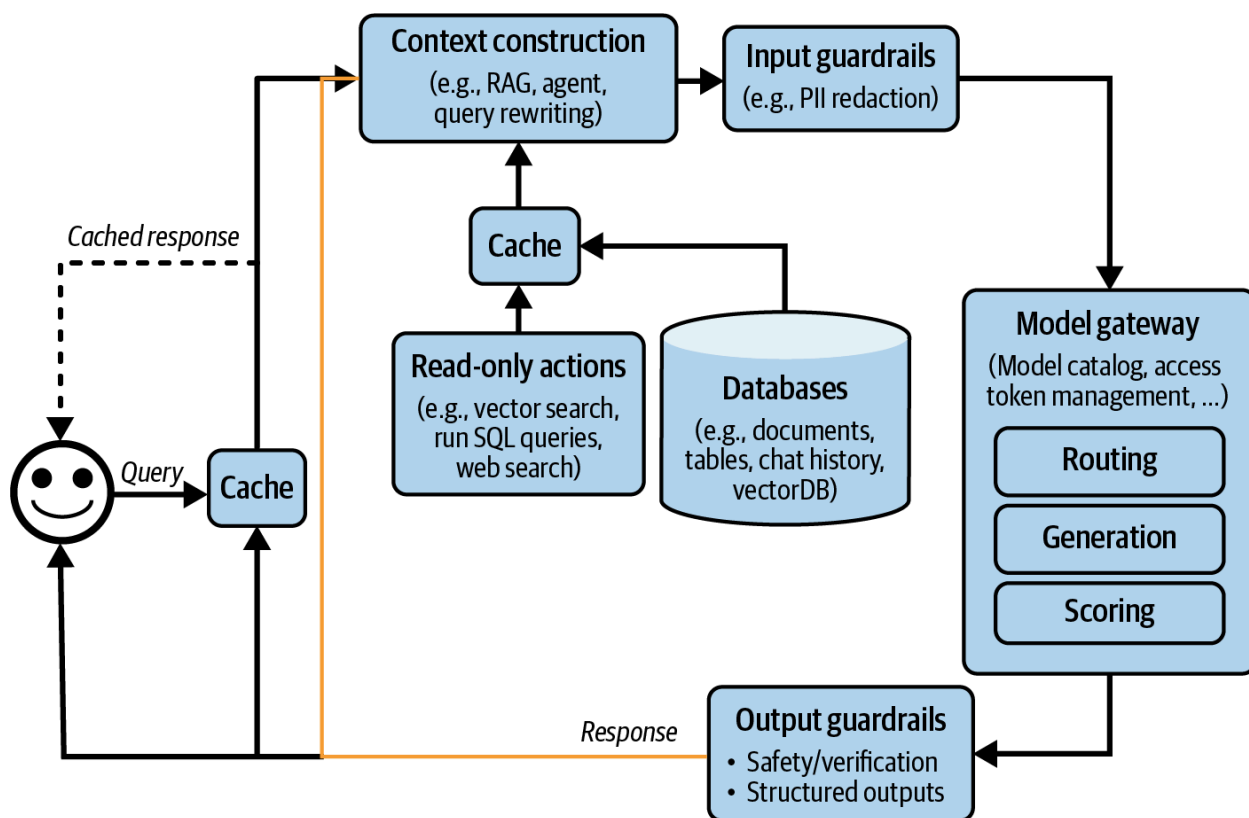
添加了缓存系统后，平台如图10-8所示。KV缓存和提示缓存通常由模型API提供商实现，因此它们未在此图像中显示。要可视化它们，我会将它们放在模型API框中。有一个新箭头将生成的响应添加到缓存中。



<图10-8的描述: 添加了缓存的AI应用架构>

步骤5. 添加代理模式

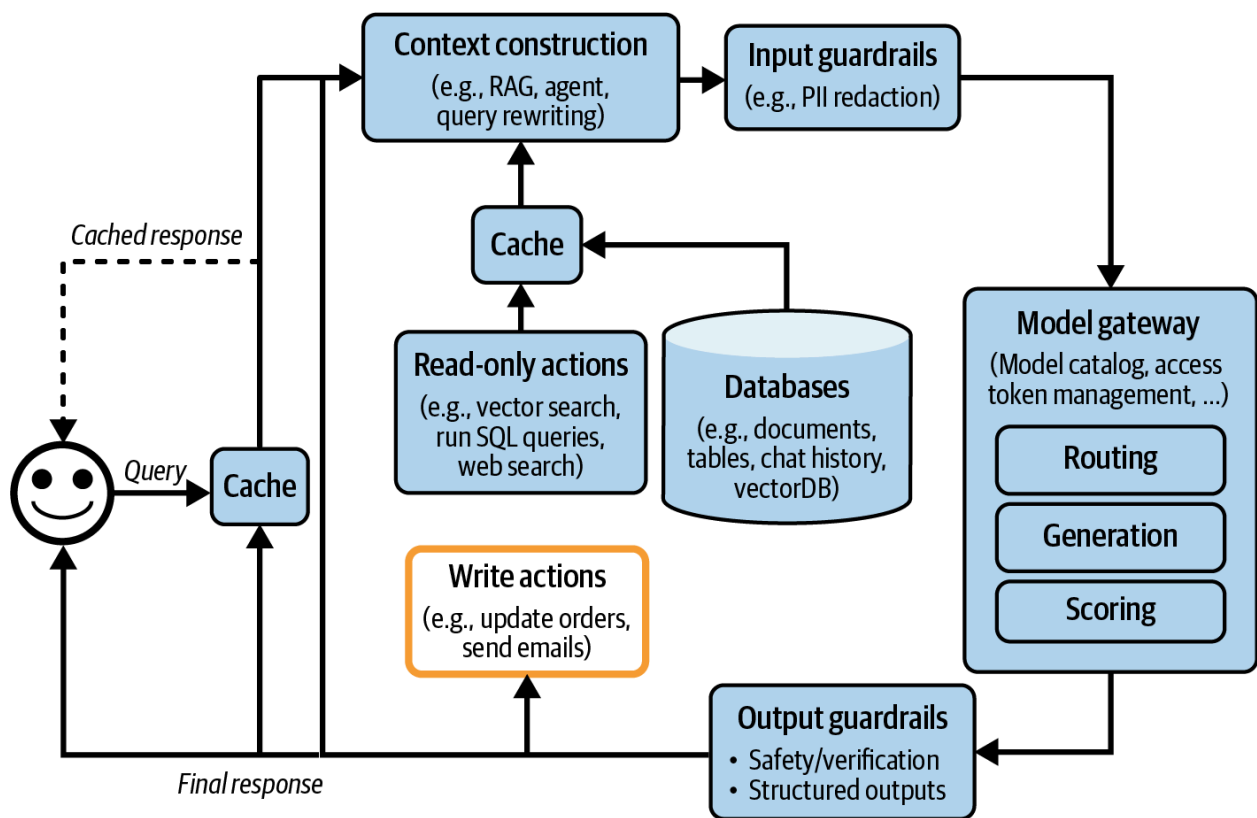
到目前为止讨论的应用仍然相当简单。每个查询都遵循顺序流程。然而，如第6章所讨论的，应用流程可能更复杂，具有循环、并行执行和条件分支。第6章讨论的代理模式可以帮助你构建复杂的应用。例如，系统生成输出后，它可能确定尚未完成任务，需要执行另一次检索以收集更多信息。原始响应与新检索的上下文一起被传递到同一个模型或不同的模型中。这创建了一个循环，如图10-9所示。



<图10-9的描述:黄色箭头允许生成的响应反馈回系统, 实现更复杂的应用模式>

模型的输出也可以用于调用写入操作, 例如撰写电子邮件、下订单或初始化银行转账。写入操作允许系统直接对其环境进行更改。如第6章所讨论的, 写入操作可以使系统更加强大, 但也会使其面临更多风险。给模型提供写入操作的访问权限应当非常谨慎。添加了写入操作后, 架构如图10-10所示。

如果你已经遵循了到目前为止的所有步骤, 你的架构可能已经变得相当复杂。虽然复杂的系统可以解决更多任务, 但它们也引入了更多的失败模式, 由于有许多潜在的失败点, 使它们更难调试。下一节将介绍改善系统可观测性的最佳实践。



<图10-10的描述:使系统能够执行写入操作的应用架构>

监控和可观测性

尽管我将可观测性放在独立的部分，但可观测性应该是产品设计的组成部分，而不是事后考虑。产品越复杂，可观测性就越关键。

可观测性是所有软件工程学科中的普遍实践。这是一个拥有既定最佳实践和许多现成专有和开源解决方案的大型行业。为避免重新发明轮子，我将专注于基于基础模型构建的应用的独特之处。本书的GitHub仓库包含资源，供那些想了解更多关于可观测性的人使用。

监控的目标与评估的目标相同：减轻风险并发现机会。监控应该帮助你减轻的风险包括应用故障、安全攻击和漂移。监控可以帮助发现应用改进和成本节约的机会。监控还可以通过提供系统性能的可见性来帮助你保持责任感。

三个指标可以帮助评估系统可观测性的质量，这些指标源于DevOps社区：

- **MTTD(平均检测时间)**：当发生不好的事情时，需要多长时间才能检测到？
- **MTTR(平均响应时间)**：检测到后，需要多长时间才能解决？
- **CFR(变更失败率)**：导致需要修复或回滚的故障的变更或部署的百分比。如果你不知道你的CFR，是时候重新设计你的平台，使其更易于观察。

CFR高并不一定表示监控系统不好。然而，你应该重新思考你的评估流程，使得不良变更在部署前被捕获。评估和监控需要紧密合作。评估指标应该很好地转化为监控指标，这意味着在评估期间表现良好的模型在监控期间也应该表现良好。监控期间检测到的问题应该反馈给评估流程。

监控与可观测性

自2010年代中期以来，行业开始使用"可观测性"一词而非"监控"。监控对系统内部状态与其输出之间的关系没有假设。你监控系统的外部输出，以了解系统内部何时出错——没有保证外部输出能帮助你找出什么出错了。

另一方面，可观测性比传统监控做出更强的假设：系统的内部状态可以从其外部输出的知识中推断出来。当一个可观察系统出现问题时，我们应该能够通过查看系统的日志和指标来弄清楚出了什么问题，而无需向系统发送新代码。可观测性是关于以确保收集和分析有关系统运行时的足够信息的方式来设置你的系统，这样当出现问题时，它可以帮助你找出问题所在。

在本书中，我将使用"监控"一词指代跟踪系统信息的行为，而"可观测性"指代设置、跟踪和调试系统的整个过程。

指标

讨论监控时，大多数人会想到指标。然而，指标本身不是目标。坦率地说，大多数公司不关心你的应用的输出相关性分数是多少，除非它服务于一个目的。指标的目的在于告诉你什么时候出了问题，并识别改进的机会。

在列出要跟踪的指标之前，重要的是了解你想要捕获的失败模式，并围绕这些失败设计指标。例如，如果你不希望你的应用产生幻觉，设计有助于检测幻觉的指标。一个相关指标可能是应用的输出是否可以从上下文中推断出来。如果你不希望你的应用消耗你的API额度，跟踪与API成本相关的指标，例如每个请求的输入和输出令牌数量，或你的缓存成本和命中率。

由于基础模型可以生成开放式输出，事情可能以多种方式出错。指标设计需要分析思维、统计知识，以及通常的创造力。应该跟踪哪些指标高度依赖于具体应用。

本书已经涵盖了许多不同类型的模型质量指标(第4-6章，以及本章稍后部分)和许多不同的计算方法(第3章和第5章)。在这里，我将快速复习一下。

最容易跟踪的失败类型是格式失败，因为它们容易被发现和验证。例如，如果你期望JSON输出，跟踪模型输出无效JSON的频率，以及在无效JSON输出中，有多少可以轻松修复(缺少一个闭合括号容易修复，但缺少预期的键更难)。

对于开放式生成，考虑监控事实一致性和相关生成质量指标，如简洁性、创造性或积极性。这些指标中的许多可以使用AI评判来计算。

如果安全是一个问题，你可以跟踪与毒性相关的指标，并在输入和输出中检测私人和敏感信息。跟踪你的护栏被触发的频率以及你的系统拒绝回答的频率。也要检测对你系统的异常查询，因为它们可能揭示有趣的边缘情况或提示攻击。

模型质量也可以通过用户自然语言反馈和对话信号推断。例如，一些你可以跟踪的简单指标包括：

- 用户中途停止生成的频率是多少？
- 每次对话的平均转折次数是多少？
- 每个输入的平均令牌数是多少？用户是否在使用你的应用处理更复杂的任务，还是他们正在学习使提示更简洁？
- 每个输出的平均令牌数是多少？某些模型比其他模型更冗长吗？某些类型的查询更可能导致冗长的答案吗？
- 模型的输出令牌分布是什么？它随时间如何变化？模型是否变得更加多样化或更少多样化？

与长度相关的指标对于跟踪延迟和成本也很重要，因为更长的上下文和响应通常会增加延迟并产生更高的成本。

应用管道中的每个组件都有自己的指标。例如，在RAG应用中，检索质量通常使用上下文相关性和上下文精度来评估。向量数据库可以通过评估索引数据需要多少存储空间以及查询数据需要多长时间来评估。

鉴于你可能有多个指标，衡量这些指标如何相互关联，特别是与你的业务北极星指标的关联，这很有用。北极星指标可以是DAU（日活跃用户）、会话持续时间（用户积极参与应用的时间长度）或订阅。与你的北极星强烈相关的指标可能会给你关于如何改进北极星的想法。完全不相关的指标也可能给你关于不需要优化什么的想法。

跟踪延迟对于理解用户体验至关重要。常见的延迟指标，如第9章所讨论的，包括：

- 首个令牌时间(TTFT)：生成第一个令牌所需的时间。
- 每个输出令牌时间(TPOT)：生成每个输出令牌所需的时间。
- 总延迟：完成响应所需的总时间。

跟踪每个用户的所有这些指标，以了解你的系统如何随着更多用户而扩展。

你还需要跟踪成本。与成本相关的指标是查询数量和输入和输出令牌的数量，如每秒令牌数(TPS)。如果你使用有速率限制的API，跟踪每秒请求数很重要，以确保你保持在分配的限制内，避免潜在的服务中断。

在计算指标时，你可以选择抽样检查和穷尽检查。抽样检查涉及对数据子集进行采样，以快速识别问题，而穷尽检查评估每个请求，以获得全面的性能视图。选择取决于你的系统要求和可用资源，两者结合可提供平衡的监控策略。

在计算指标时，确保它们可以按相关轴分解，如用户、发布、提示/链版本、提示/链类型和时间。这种粒度有助于理解性能变化并识别特定问题。

日志和跟踪

指标通常是聚合的。它们浓缩了随时间发生在你的系统中的事件的信息。它们帮助你一目了然地了解你的系统表现如何。然而，有很多问题是指标无法帮助你回答的。例如，在看到特定活动的峰值后，你可能想知道：“这以前发生过吗？”日志可以帮助你回答这个问题。

如果指标是代表属性和事件的数值测量，日志是事件的仅附加记录。在生产中，调试过程可能如下所示：

1. 指标告诉你五分钟前出了问题，但它们没有告诉你发生了什么。
2. 你查看大约五分钟前发生的事件的日志，以找出发生了什么。
3. 将日志中的错误与指标相关联，以确保你已经确定了正确的问题。

为了快速检测，指标需要快速计算。为了快速响应，日志需要随时可用和可访问。如果你的日志延迟15分钟，你将不得不等待日志到达，才能追踪5分钟前发生的问题。

因为你不确切知道将来需要查看什么日志，日志记录的一般规则是记录一切。记录所有配置，包括模型API端点、模型名称、采样设置（温度、top-p、top-k、停止条件等）和提示模板。

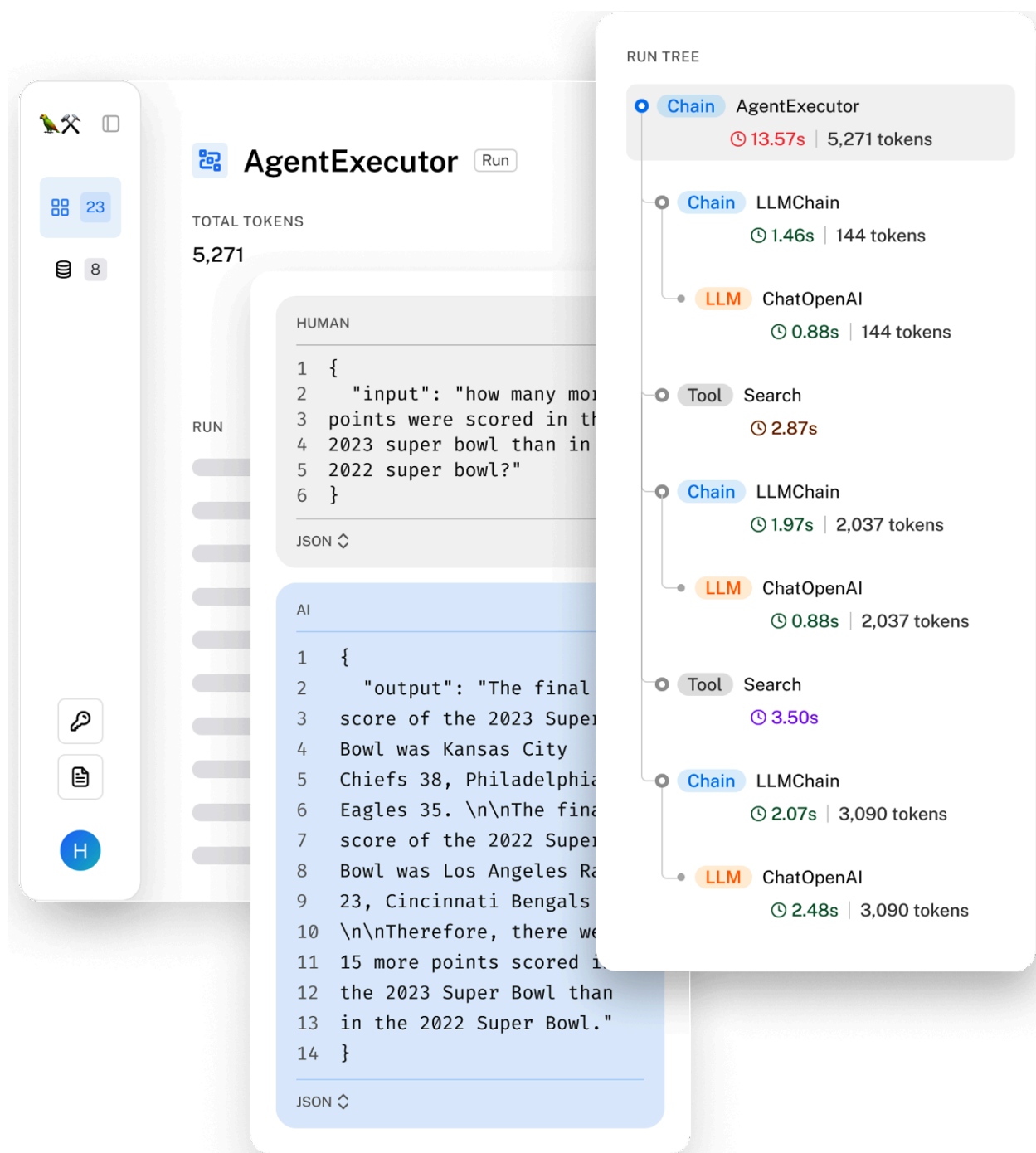
记录用户查询、发送给模型的最终提示、输出和中间输出。记录它是否调用任何工具。记录工具输出。记录组件何时开始、结束，何时崩溃等。记录日志片段时，确保给它添加标签和ID，这些可以帮助你了解这个日志来自系统的哪个部分。

记录一切意味着你拥有的日志量可能会迅速增长。许多用于自动日志分析和日志异常检测的工具都由AI提供支持。

虽然手动处理日志是不可能的，但每天手动检查你的生产数据以了解用户如何使用你的应用是有用的。Shankar等人(2024)发现，开发者对什么构成好坏输出的看法随着他们与更多数据的交互而变化，使他们能够重写提示以增加好响应的机会，并更新评估流程以捕获坏响应。

如果日志是一系列不连贯的事件，跟踪是通过将相关事件链接在一起形成的完整事务或过程的时间线，显示从开始到结束每个步骤是如何连接的。简而言之，跟踪是对请求在各个系统组件和服务中执行路径的详细记录。在AI应用中，跟踪揭示了从用户发送查询到返回最终响应的整个过程，包括系统采取的操作、检索的文档和发送给模型的最终提示。它还应显示每个步骤花费了多少时间及其相关成本（如可测量）。图10-11是LangSmith中请求跟踪的可视化。

理想情况下，你应该能够逐步跟踪每个查询通过系统的转换。如果查询失败，你应该能够确定它出错的确切步骤：是否处理不正确，检索到的上下文不相关，或者模型生成了错误的响应。



<图10-11的描述:LangSmith可视化的请求跟踪>

漂移检测

系统组件越多，可能变化的东西就越多。在AI应用中，这些可能是：

系统提示变化

你的应用系统提示可能在你不知情的情况下变化的原因有很多。系统提示可能是基于提示模板构建的, 而该提示模板已更新。同事可能发现了拼写错误并进行了修复。一个简单的逻辑应该足以捕获你的应用系统提示何时发生变化。

用户行为变化

随着时间的推移, 用户调整他们的行为以适应技术。例如, 人们已经弄清楚如何构建他们的查询以在Google搜索上获得更好的结果, 或者如何使他们的文章在搜索结果中排名更高。居住在有自动驾驶汽车的地区的人已经弄明白如何“欺负”自动驾驶汽车, 使其让路(Liu等人, 2020)。你的用户可能会改变他们的行为, 以从你的应用中获得更好的结果。例如, 你的用户可能学会编写指令, 使响应更简洁。这可能导致响应长度随时间逐渐下降。如果你只看指标, 可能不容易看出是什么导致了这种逐渐下降。你需要调查来了解根本原因。

底层模型变化

通过API使用模型时, API可能保持不变, 而底层模型被更新。如第4章所述, 模型提供商可能并不总是披露这些更新, 让你来检测任何变化。同一API的不同版本可能对性能产生重大影响。例如, Chen等人(2023)观察到GPT-4和GPT-3.5的2023年3月版本与2023年6月版本在基准分数上有显著差异。同样, Voiceflow报告从较旧的GPT-3.5-turbo-0301切换到较新的GPT-3.5-turbo-1106时, 性能下降了10%。

AI管道编排

AI应用可能相当复杂, 由多个模型组成, 从许多数据库检索数据, 并能访问各种工具。编排器帮助你指定这些不同组件如何一起工作, 创建端到端管道。它确保数据在组件之间无缝流动。在高层次上, 编排器分两个步骤操作, 组件定义和链接:

组件定义

你需要告诉编排器你的系统使用什么组件, 包括不同的模型、用于检索的外部数据源以及你的系统可以使用的工具。模型网关可以使添加模型变得更容易。你还可以告诉编排器是否使用任何评估和监控工具。

链接

链接基本上是函数组合: 它将不同的函数(组件)组合在一起。在链接(管道)中, 你告诉编排器从接收用户查询到完成任务的系统步骤。以下是步骤示例:

1. 处理原始查询。
2. 基于处理后的查询检索相关数据。
3. 将原始查询和检索到的数据组合, 创建符合模型预期格式的提示。
4. 模型根据提示生成响应。
5. 评估响应。
6. 如果响应被认为是好的, 将其返回给用户。如果不是, 将查询路由给人工操作员。

编排器负责在组件之间传递数据。它应该提供工具，帮助确保当前步骤的输出符合下一步骤期望的格式。理想情况下，它应该在数据流由于错误(如组件故障或数据不匹配故障)而中断时通知你。

警告

AI管道编排器与一般工作流编排器(如Airflow或Metaflow)不同。

在设计具有严格延迟要求的应用的管道时，尝试尽可能多地并行执行。例如，如果你有一个路由组件(决定将查询发送到哪里)和一个PII移除组件，两者可以同时完成。

有许多AI编排工具，包括LangChain、LlamaIndex、Flowise、Langflow和Haystack。由于检索和工具使用是常见的应用模式，许多RAG和代理框架也是编排工具。

虽然在开始项目时直接跳到编排工具很诱人，但你可能想先不使用它来构建你的应用。任何外部工具都会带来额外的复杂性。编排器可能会抽象掉系统工作方式的关键细节，使理解和调试你的系统变得困难。

随着你进入应用开发过程的后期阶段，你可能会决定编排器可以使你的生活更轻松。评估编排器时，请记住以下三个方面：

集成和可扩展性

评估编排器是否支持你已经使用或将来可能采用的组件。例如，如果你想使用Llama模型，检查编排器是否支持它。鉴于有多少模型、数据库和框架，编排器不可能支持所有东西。因此，你还需要考虑编排器的可扩展性。如果它不支持特定组件，更改有多难？

支持复杂管道

随着你的应用变得更加复杂，你可能需要管理涉及多个步骤和条件逻辑的复杂管道。支持高级功能(如分支、并行处理和错误处理)的编排器将帮助你有效管理这些复杂性。

易用性、性能和可扩展性

考虑编排器的用户友好性。寻找直观的API、全面的文档和强大的社区支持，因为这些可以显著减少你和你的团队的学习曲线。避免向你的应用引入隐藏API调用或延迟的编排器。此外，确保编排器随着应用数量、开发人员和流量的增长而有效扩展。

用户反馈

用户反馈在软件应用中一直扮演关键角色，主要有两种方式：评估应用性能和指导其开发。然而，在AI应用中，用户反馈扮演着更为重要的角色。用户反馈是专有数据，而数据是竞争优势。设计良好的用户反馈系统是创建第8章讨论的数据飞轮所必需的。

用户反馈不仅可用于个性化为单个用户定制模型，还可用于训练未来的模型迭代。随着数据变得越来越稀缺，专有数据比以往任何时候都更有价值。快速推出并吸引早期用户的产品可以收集数据，持续改进模型，使竞争对手难以赶上。

重要的是要记住，用户反馈是用户数据。利用用户反馈需要与利用任何数据时相同的谨慎。应尊重用户隐私。用户有权知道他们的数据如何被使用。

提取对话反馈

传统上，反馈可以是显式的或隐式的。显式反馈是用户响应应用中明确反馈请求提供的信息，如点赞/点踩、投票/反对、星级评价，或对“我们是否解决了你的问题？”问题的是/否回答。显式反馈在应用之间相当标准——询问一个人是否喜欢某物的方式只有这么多。因此，显式反馈被更好地理解。

隐式反馈是从用户行为中推断出的信息。例如，如果有人购买向他们推荐的产品，这意味着这是一个好的推荐。什么可以被视为隐式反馈取决于用户可以在每个应用中做什么操作，因此高度依赖于应用。基础模型使一个全新的应用世界成为可能，伴随着许多种隐式反馈。

许多AI应用使用的对话界面使用户更容易提供反馈。用户可以鼓励好的行为并纠正错误，就像他们在日常对话中提供反馈一样。用户用来向AI提供指导的语言可以传达关于应用性能和用户偏好的反馈。

例如，假设你正在使用AI助手帮助你计划去澳大利亚的旅行。你要求AI找一家悉尼的酒店住三晚。它回应了三个建议如下：

Here are three hotel suggestions in Sydney for a 3-night stay:

1. Historic Rocks Boutique Hotel (The Rocks)

-Price: ~\$400/night

-Neighborhood: Charming streets and close to iconic sights.

2. Stylish Surry Hills House Hotel (Surry Hills)

-Price: ~\$200/night

-Neighborhood: Trendy, with vibrant cafes and art galleries.

3. Chill Bondi Beachside Hotel (Bondi Beach)

-Price: ~\$300/night

Neighborhood: Beachside, ideal for relaxation and city exploration.

你对这三个建议的回应揭示了你的偏好。例如，如果你回应“是的，给我预订靠近画廊的那个”，你表明了对艺术的兴趣。另一方面，回应“有没有低于\$200的？”揭示了一个注重价格的偏好，并表明助手还没有完全了解你。

从对话中提取的用户反馈可用于评估、开发和个性化：

- 评估：得出指标来监控应用
- 开发：训练未来的模型或指导其开发
- 个性化：为每个用户个性化应用

隐式对话反馈可以从用户消息的内容和他们的沟通模式中推断。因为反馈融入到日常对话中，所以提取它也具有挑战性。虽然关于对话提示的直觉可以帮助你设计初始信号集，但需要严格的数据分析和用户研究来理解。

虽然对话反馈因ChatGPT的流行而受到更多关注，但在ChatGPT出现之前几年，它已经是一个活跃的研究领域。强化学习社区自2010年代末以来一直试图让RL算法从自然语言反馈中学习，其中许多取得了有希望的结果；参见Fu等人(2019)；Goyal等人(2019)；Zhou和Small(2020)；以及Sumers等人(2020)。自然语言反馈对于早期对话式AI应用也非常有兴趣，如Amazon Alexa（Ponnusamy等人，2019；Park等人，2020），Spotify的语音控制功能（Xiao等人，2021）和Yahoo! Voice（Hashimoto和Sassano，2018）。

自然语言反馈

从消息内容中提取的反馈称为自然语言反馈。以下是一些告诉你对话进展如何的自然语言反馈信号。在生产中跟踪这些信号对于监控你的应用性能很有用。

提前终止

如果用户提前终止响应，例如，在响应生成一半时停止，退出应用（对于网络和移动应用），告诉模型停止（对于语音助手），或者只是让代理悬着（例如，不向代理响应你想要它继续哪个选项），可能表明对话进展不顺利。

错误纠正

如果用户以“不，...”或“我的意思是，...”开始他们的后续，模型的响应可能偏离了目标。

为了纠正错误，用户可能尝试重新表述他们的请求。图10-12显示了用户尝试纠正模型误解的例子。可以使用启发式或ML模型检测重新表述尝试。

weather in mission bay today



Searched 3 sites

Today in Mission Bay, San Francisco, the weather is mostly sunny with temperatures ranging from 56°F to 68°F (13°C to 20°C). Winds are expected

Rephrase with clarifying information



mission bay san diego



Searched 4 sites

Today in Mission Bay, San Diego, the weather is partly cloudy with patchy fog in the morning, becoming mostly sunny later in the day. Temperatures are expected to range between 68°F and 74°F (20°C to 23°C). Winds will be light, becoming westward at around 10 mph in the afternoon. Overnight, the skies will be mostly clear, turning partly cloudy with some patchy fog, and temperatures will drop to between 63°F and 68°F (17°C to 20°C).

<图10-12的描述: 因为用户既提前终止生成又重新表述问题, 可以推断模型误解了原始请求的意图>

用户还可以指出模型应该做的具体不同之处。例如, 如果用户要求模型总结一个故事, 而模型混淆了一个角色, 这个用户可以给出反馈, 如:"比尔是嫌疑人, 不是受害者。"模型应该能够接受这个反馈并修改摘要。

这种行动纠正反馈在代理用例中特别常见, 用户可能会引导代理采取更可选的行动。例如, 如果用户分配代理进行关于XYZ公司的市场分析, 这个用户可能会给出反馈, 如"你还应该查看XYZ的GitHub页面"或"查看CEO的X个人资料"。

有时, 用户可能希望模型通过要求明确确认来纠正自己, 如"你确定吗?", "再检查一下", 或"给我看看来源"。这并不一定意味着模型给出了错误的答案。然而, 它可能意味着你的模型的答案缺乏用户正在寻找的细节。它也可能表明对你的模型的普遍不信任。

一些应用让用户直接编辑模型的响应。例如, 如果用户要求模型生成代码, 而用户纠正生成的代码, 这是一个很强的信号, 表明被编辑的代码不太对。

用户编辑也是偏好数据的宝贵来源。回想一下, 偏好数据通常以(查询、获胜响应、失败响应)的格式出现, 可用于将模型与人类偏好对齐。每次用户编辑构成一个偏好示例, 原始生成的响应是失败响应, 编辑后的响应是获胜响应。

投诉

通常，用户只是抱怨你的应用的输出，而不尝试纠正它们。例如，他们可能抱怨回答错误、不相关、有毒、冗长、缺乏细节或只是不好。表10-1显示了通过对FITS(互动对话与搜索反馈)数据集(Xu等人, 2022)进行自动聚类得出的八组自然语言反馈。

表10-1. 通过对FITS数据集自动聚类得出的反馈类型(Xu等人, 2022)。结果来自Yuan等人(2023)。

组别	反馈类型	数量	百分比
1	再次澄清他们的需求。	3702	26.54%
2	抱怨机器人(1)没有回答问题或(2)提供不相关信息或(3)要求用户自行查找答案。	2260	16.20%
3	指出可以回答问题的特定搜索结果。	2255	16.17%
4	建议机器人应该使用搜索结果。	2130	15.27%
5	指出答案是(1)事实不正确, 或(2)没有以搜索结果为依据。	1572	11.27%
6	指出机器人的回答不够具体/准确/完整/详细。	1309	9.39%
7	指出机器人对其回答不自信, 总是以"我不确定"或"我不知道"开始回应。	582	4.17%
8	抱怨机器人回应中的重复/粗鲁。	137	0.99%

了解机器人如何让用户失望对于改进它至关重要。例如，如果你知道用户不喜欢冗长的回答，你可以更改机器人的提示，使其更简洁。如果用户不满意是因为回答缺乏细节，你可以提示机器人更具体。

情感

投诉也可以是一般的负面情感表达(沮丧、失望、嘲笑等)，而不解释原因，比如"唉"。这听起来可能有些反乌托邦，但分析用户在与机器人对话过程中的情感可能让你洞察机器人的表现。一些呼叫中心在整个通话过程中跟踪用户的声音。如果用户越来越大声，说明有问题。相反，如果有人开始对话时很生气但结束时很开心，对话可能已经解决了他们的问题。

自然语言反馈也可以从模型的响应中推断。一个重要的信号是模型的拒绝率。如果模型说"抱歉, 我不知道那个"或"作为语言模型, 我不能...", 用户可能不开心。

其他对话反馈

其他类型的对话反馈可以从用户行为而不是消息中派生。

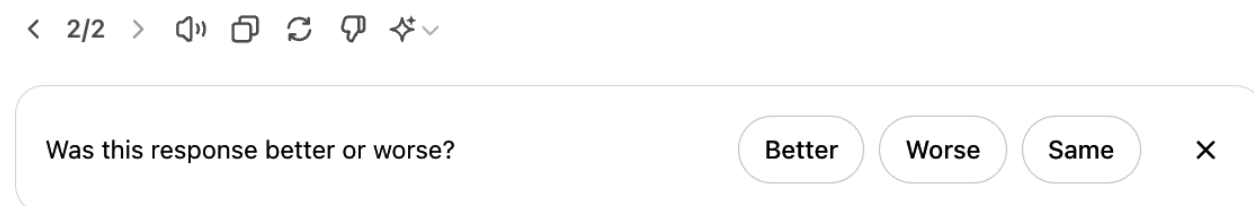
重新生成

许多应用让用户生成另一个响应, 有时使用不同的模型。如果用户选择重新生成, 可能是因为他们对第一个响应不满意。然而, 也可能是第一个响应足够好, 但用户想要选项进行比较。这在创意请求(如图像或故事生成)中特别常见。

基于使用的计费应用比订阅制应用可能有更强的重新生成信号。使用基于使用的计费, 用户不太可能出于好奇而重新生成并额外花钱。

就我个人而言, 我经常为复杂请求选择重新生成, 以确保模型的响应一致。如果两个响应给出相互矛盾的答案, 我就不能信任任何一个。

重新生成后, 一些应用可能明确要求比较新响应与之前响应, 如图10-13所示。这种更好或更差的数据, 再次, 可用于偏好微调。



<图10-13的描述: 当用户重新生成另一个响应时, ChatGPT要求比较反馈>

对话组织

用户采取的组织对话的行动(如删除、重命名、分享和收藏)也可以是信号。删除对话是对话不好的很强信号, 除非这是一个尴尬的对话, 用户想要删除其痕迹。重命名对话表明对话是好的, 但自动生成的标题不好。

对话长度

另一个常被跟踪的信号是每次对话的回合数。这是积极还是消极信号取决于应用。对于AI伴侣, 长对话可能表明用户享受对话。然而, 对于面向生产力的聊天机器人(如客户支持), 长对话可能表明机器人在帮助用户解决问题方面效率低下。

对话多样性

对话长度也可以与对话多样性一起解释, 对话多样性可以通过不同标记或主题计数来衡量。例如, 如果对话很长, 但机器人只是重复几行, 用户可能陷入循环。

显式反馈更容易解释，但它需要用户额外的努力。由于许多用户可能不愿意付出这额外的工作，显式反馈可能很稀疏，特别是在用户群较小的应用中。显式反馈也受到响应偏差的影响。例如，不满意的用户可能更可能抱怨，导致反馈看起来比实际情况更负面。

隐式反馈更丰富——什么可以被视为隐式反馈仅受你的想象力限制——但它更嘈杂。解释隐式信号可能具有挑战性。例如，分享对话可能是负面或积极信号。例如，我的一个朋友主要在模型犯了一些明显错误时分享对话，而另一个朋友主要将有用的对话与他们的同事分享。了解你的用户为什么执行每个操作很重要。

添加更多信号可以帮助澄清意图。例如，如果用户在分享链接后重新表述他们的问题，这可能表明对话没有满足他们的期望。从对话中提取、解释和利用隐式响应是一个小但不断增长的研究领域。

反馈设计

如果你不确定收集什么反馈，我希望上一节给了你一些想法。

本节讨论何时以及如何收集这些宝贵的反馈。

何时收集反馈

反馈可以且应该在整个用户旅程中收集。用户应该有选择提供反馈的选项，特别是报告错误，无论何时需要。然而，反馈收集选项应该不引人注目。它不应该干扰用户工作流程。以下是一些用户反馈可能特别有价值的地方。

在开始时

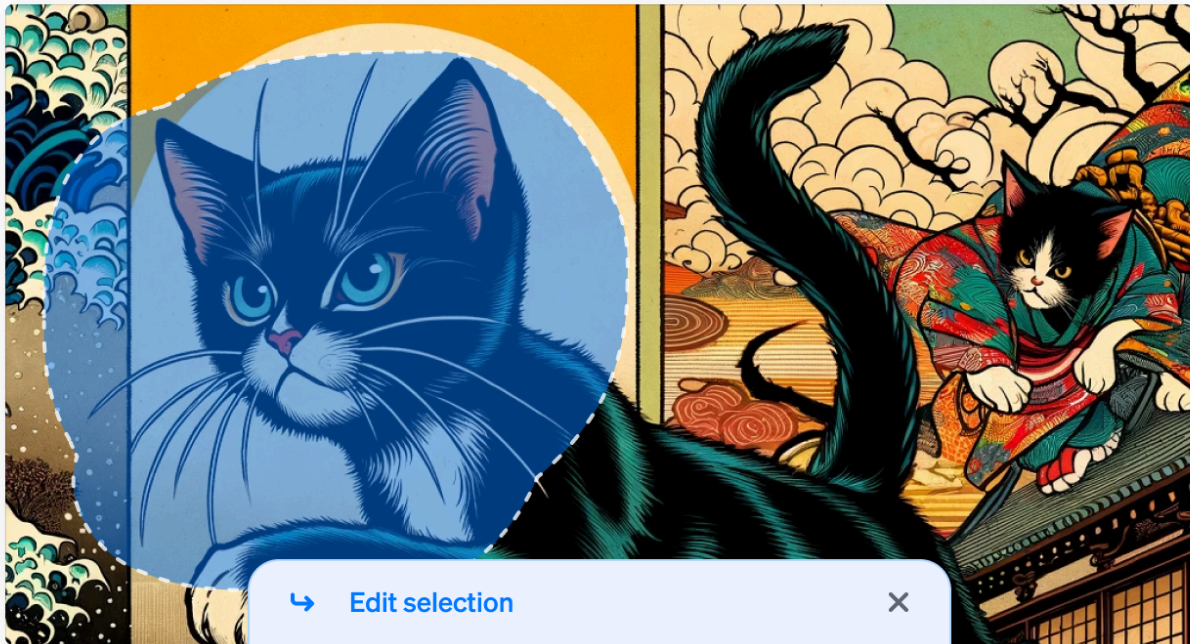
当用户刚刚注册时，用户反馈可以帮助为用户校准应用。例如，人脸ID应用首先必须扫描你的脸才能工作。语音助手可能会要求你大声朗读一个句子，以识别你的唤醒词的声音（激活语音助手的词，如“嘿Google”）。语言学习应用可能会问你几个问题来评估你的技能水平。对于某些应用，如人脸ID，校准是必要的。然而，对于其他应用，初始反馈应该是可选的，因为它为用户尝试你的产品创造了摩擦。如果用户没有指定他们的偏好，你可以回退到中性选项，并随着时间的推移进行校准。

当发生不好的事情时

当模型产生幻觉响应、阻止合法请求、生成令人不安的图像或响应时间过长时，用户应该能够通知你这些失败。你可以给用户选项，对响应进行踩踏，用同一模型重新生成，或更换到另一个模型。用户可能只是给出对话反馈，如“你错了”，“太老套了”，或“我想要更短的东西”。

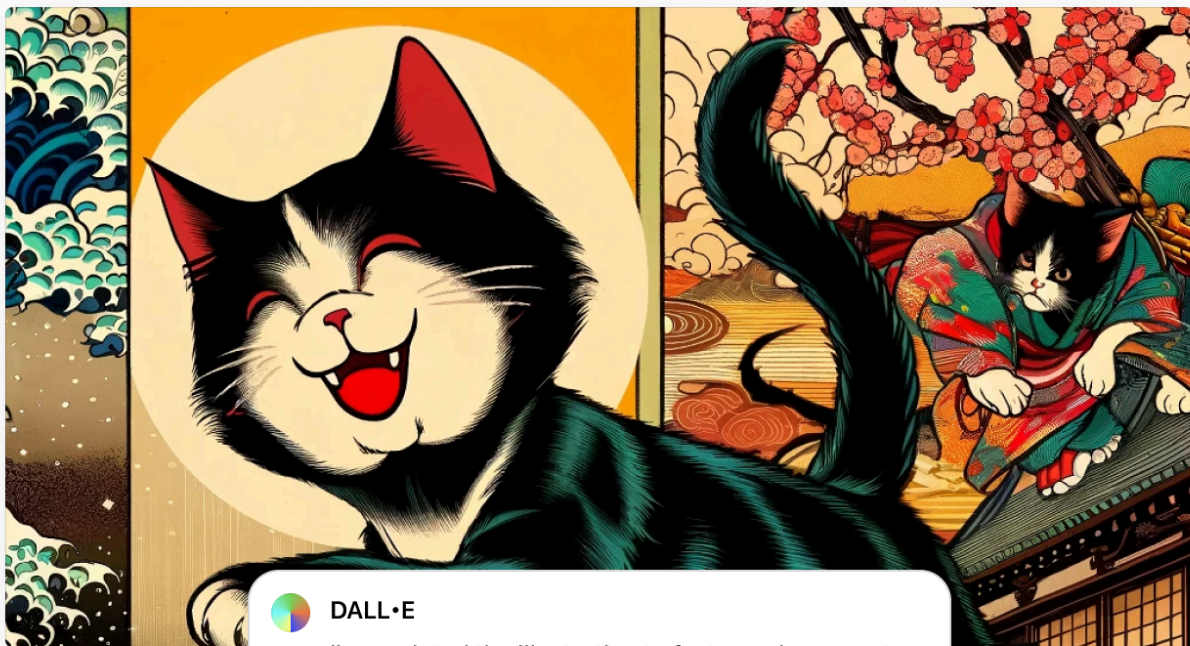
理想情况下，当你的产品出错时，用户仍然应该能够完成他们的任务。例如，如果模型错误地分类了一个产品，用户可以编辑类别。让用户与AI协作。如果不行，让他们与人类协作。许多客户支持机器人提供在对话拖延或用户看起来沮丧时将用户转接给人工代理的选项。

人工-AI协作的一个例子是图像生成的修补功能。如果生成的图像不完全符合用户需求，他们可以选择图像的一个区域，并用提示描述如何使其更好。图10-14显示了DALL-E (OpenAI, 2021) 中修补的例子。这个功能允许用户获得更好的结果，同时为开发者提供高质量的反馈。



↩ Edit selection ✕

📎 change the cat's expression to happy



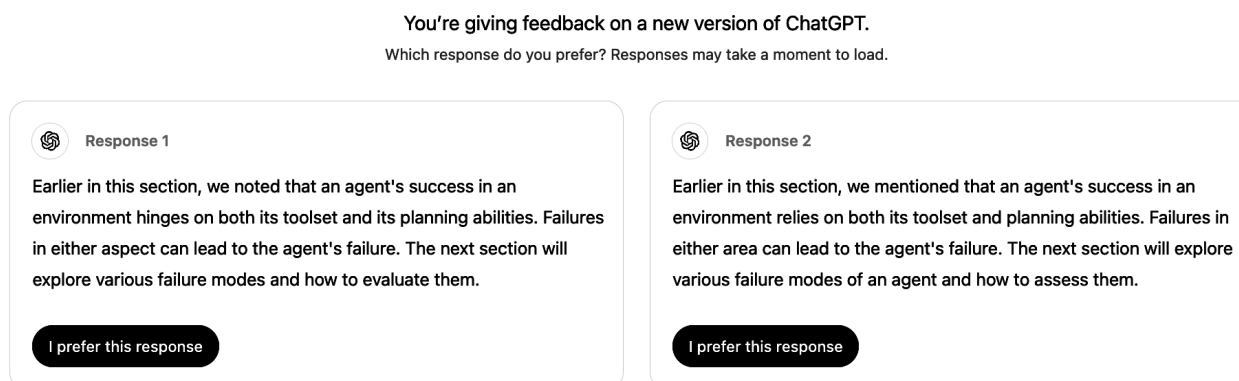
DALL·E

I've updated the illustration to feature a happy cat amid cherry blossoms, keeping with the minimalist blend of Ukiyo-e and comic book styles.

<图10-14的描述: DALL-E中修补工作原理的示例。图片由OpenAI提供>

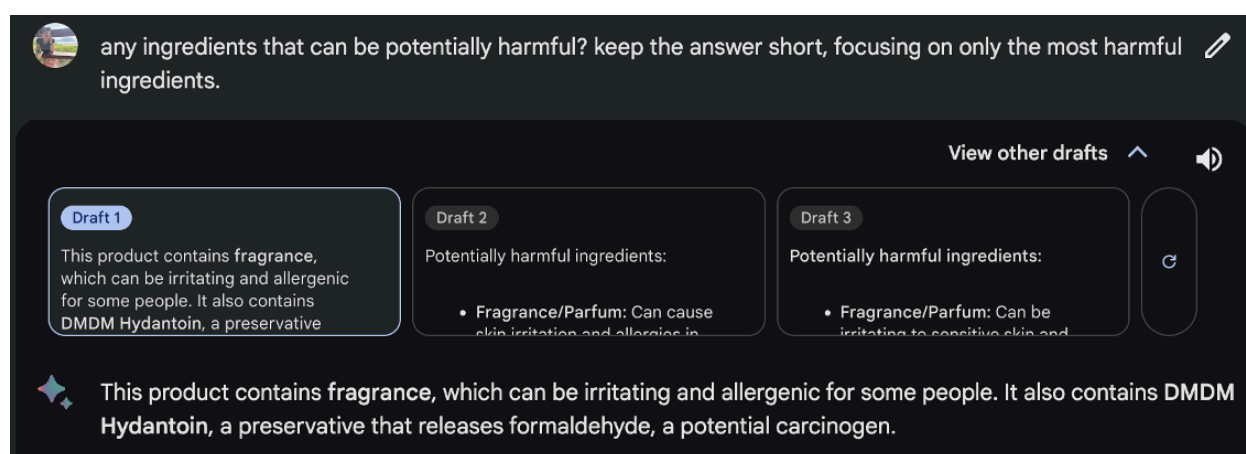
当模型信心不足时

当模型对行动不确定时, 你可以询问用户的反馈以增加它的信心。例如, 对于总结论文的请求, 如果模型不确定用户是否更喜欢简短的高层次总结还是详细的逐节总结, 模型可以并排输出两种总结。假设生成两个总结不会增加用户的延迟。用户可以选择他们更喜欢哪一个。这种比较信号可用于偏好微调。图10-15显示了生产中比较评估的例子。



<图10-15的描述: 两个ChatGPT响应的并排比较>

显示两个完整响应让用户选择意味着向该用户请求显式反馈。用户可能没有时间阅读两个完整响应或足够关心提供深思熟虑的反馈。这可能导致嘈杂的投票。一些应用, 如Google Gemini, 只显示每个响应的开头, 如图10-16所示。用户可以点击展开他们想阅读的响应。然而, 目前尚不清楚显示完整还是部分并排响应是否能提供更可靠的反馈。



<图10-16的描述: Google Gemini并排显示部分响应以进行比较反馈。用户必须点击他们想了解更多的响应, 这提供了关于他们认为哪个响应更有希望的反馈>

另一个例子是自动为你的照片添加标签的照片组织应用，这样它可以响应“显示我所有X的照片”这样的查询。当不确定两个人是否相同时，它可以向你询问反馈，如Google Photos在图10-17中所做的那样。

Same or different person?



Same



Different



Not sure

<图10-17的描述：Google Photos在不确定时询问用户反馈。两个猫图像由ChatGPT生成>

你可能想知道：当好事发生时的反馈怎么样？用户可以采取的表达满意的行动包括点赞、收藏或分享。然而，苹果的人机界面指南警告不要同时要求积极和消极反馈。你的应用应该默认生成好的结果。要求对好结果的反馈可能会给用户留下好结果是例外的印象。最终，如果用户满意，他们会继续使用你的应用。

然而，我交谈过的许多人认为用户应该有在遇到令人惊叹的事情时给予反馈的选项。一个流行的AI驱动产品的产品经理提到，他们的团队需要积极反馈，因为它揭示了用户喜爱到足以给予热情反馈的功能。这使团队能够专注于改进一小部分有高影响力的功能，而不是将资源分散在许多影响最小的功能上。

有些人避免要求积极反馈是因为担心它可能会使界面混乱或惹恼用户。然而，通过限制反馈请求的频率，可以管理这种风险。例如，如果你有大量用户群，一次只向1%的用户显示请求可以帮助收集足够的反馈，而不会干扰大多数用户的体验。请记住，要求的用户百分比越小，反馈偏见的风险越大。尽管如此，只要有足够大的用户池，反馈仍然可以提供有意义的产品洞察。

如何收集反馈

反馈应该无缝融入用户的工作流程。用户应该能够轻松提供反馈，无需额外工作。反馈收集不应该破坏用户体验，并且应该容易被忽略。应该有激励用户提供良好反馈的机制。

一个经常被引用为良好反馈设计的例子来自图像生成应用Midjourney。对于每个提示，Midjourney生成一组(四个)图像，并给用户以下选项，如图10-18所示：

1. 为任何这些图像生成未缩放版本。
2. 为任何这些图像生成变体。
3. 重新生成。

所有这些选项都给Midjourney不同的信号。选项1和2告诉Midjourney用户认为四张照片中哪一张最有希望。选项1对所选照片给出最强烈的积极信号。选项2给出较弱的积极信号。选项3表明没有一张照片足够好。然而，用户可能选择重新生成，即使现有照片很好，只是为了看看其他可能性。

A close up picture of a grinning, winking llama in police uniform in Zootopia style that makes you want to go on an adventure with - @chiphuyen (fast)



Upscale image 1 to 4 →

U1

U2

U3

U4



← Generate another batch

Generate variations →

V1

V2

V3

V4

<图10-18的描述: Midjourney的工作流程允许应用收集隐式反馈>

像GitHub Copilot这样的代码助手可能会以比最终文本更浅的颜色显示草稿, 如图10-19所示。用户可以使用Tab键接受建议, 或者只需继续输入以忽略建议, 两者都提供反馈。


```

9 class Solution:
10     def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
11         """
12         Do not return anything, modify nums1 in-place instead.
13         """
14         < 1/2 > Accept Tab Accept Word %> → ...
15         while p1 >= 0 and p2 >= 0:
16             if nums1[p1] > nums2[p2]:
17                 nums1[p] = nums1[p1]
18                 p1 -= 1
19             else:
20                 nums1[p] = nums2[p2]
21                 p2 -= 1
22             p += 1

```

<图10-19的描述: GitHub Copilot使建议和拒绝建议变得容易>

像ChatGPT和Claude这样的独立AI应用的最大挑战之一是它们没有集成到用户的日常工作流程中,这使得收集高质量的反馈变得困难,而集成产品如GitHub Copilot则容易做到这一点。例如,如果Gmail建议一个电子邮件草稿, Gmail可以跟踪这个草稿如何被使用或编辑。然而,如果你使用ChatGPT写一封电子邮件, ChatGPT不知道生成的电子邮件是否真的被发送了。

单独的反馈可能对产品分析有帮助。例如,只看点赞/点踩信息对计算人们对你的产品满意或不满意的频率很有用。然而,要进行更深入的分析,你需要反馈周围的上下文,例如之前的5到10个对话轮次。这种上下文可以帮助你找出出了什么问题。然而,获取这种上下文可能是不可能的,除非有明确的用户同意,特别是如果上下文可能包含个人可识别信息。

因此,一些产品在其服务协议中包含条款,允许他们访问用户数据进行分析和产品改进。对于没有这些条款的应用,用户反馈可能与用户数据捐赠流程相关联,在那里用户被要求捐赠(例如,分享)他们的最近互动数据以及他们的反馈。例如,提交反馈时,你可能会被要求选中一个框,分享你最近的数据作为此反馈的上下文。

向用户解释如何使用他们的反馈可以激励他们提供更多更好的反馈。你是否使用用户的反馈来个性化产品,收集关于一般使用的统计数据,或训练新模型?如果用户担心隐私,向他们保证他们的数据不会用于训练模型或不会离开他们的设备(只有当这些是真实的)。

不要要求用户做不可能的事情。例如,如果你从用户那里收集比较信号,不要要求他们在他们不理解的两个选项之间做出选择。例如,当ChatGPT要求我在两个可能的统计问题答案之间选择时,我曾经感到困惑,如图10-20所示。我希望有一个选项让我说“我不知道”。



You

what kind of test is this

Which response do you prefer?
Your choice will help make ChatGPT better.



Response 1

The type of test depends on the context and the nature of the data, but based on the formula used, it is typically one of the following:

1. Two-Sample Z-Test for Proportions



Response 2

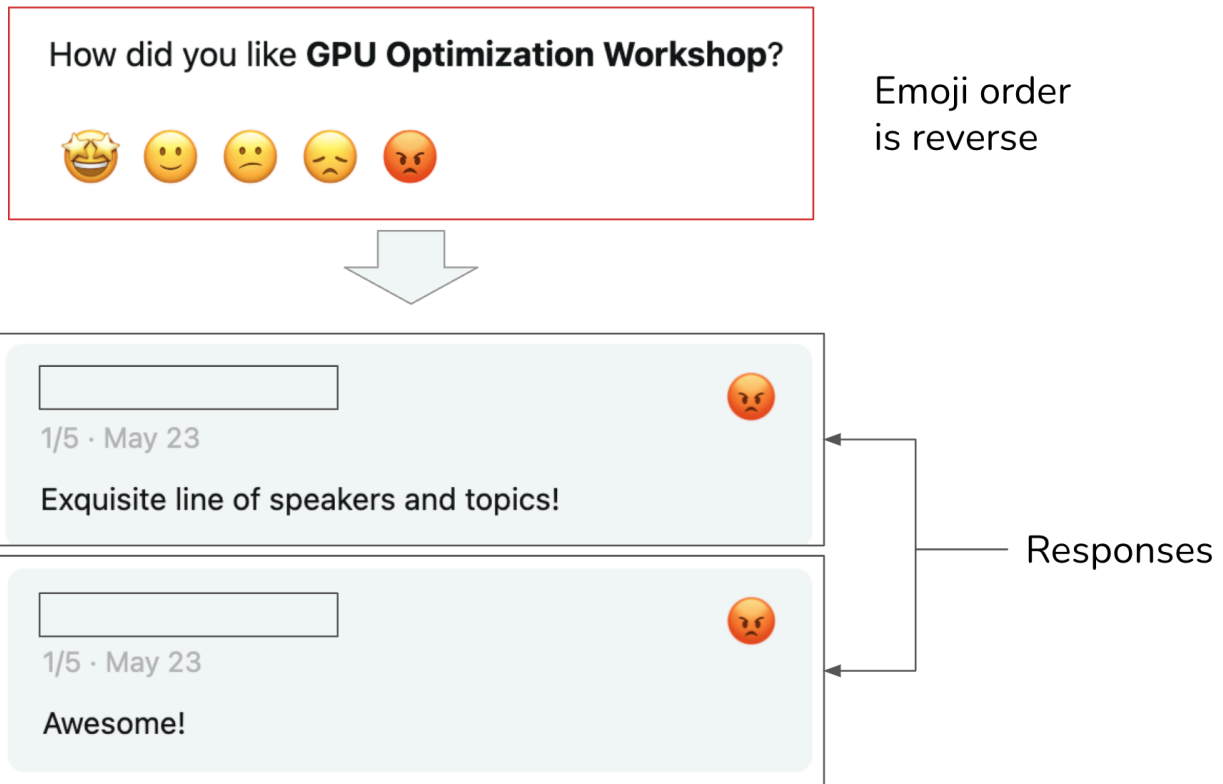
These calculations are associated with a **two-tailed Z-test**. Here's a breakdown of what that entails:

Understanding the Two-Tailed Z-Test

<图10-20的描述: ChatGPT要求用户选择他们喜欢的响应的例子。然而, 对于像这样的数学问题, 正确答案不应该是偏好的问题>

如果它们能帮助人们理解, 可以为选项添加图标和工具提示。避免可能使用户混淆的设计。模糊的说明可能导致嘈杂的反馈。我曾经使用Luma举办了一个GPU优化研讨会, 收集反馈。当我阅读负面反馈时, 我感到困惑。尽管回应是积极的, 但星级评定是1/5。当我深入研究时, 我意识到Luma在他们的反馈收集表单中使用了表情符号来表示数字, 但愤怒的表情符号, 对应于一星评级, 被放在了五星评级应该在的地方, 如图10-21所示。

要注意你是否希望用户的反馈是私人的还是公开的。例如, 如果用户喜欢某事, 你是否希望这信息显示给其他用户? 在早期, Midjourney的反馈——有人选择放大图像、生成变体或重新生成另一批图像——是公开的。



<图10-21的描述：因为Luma将愤怒的表情符号（对应于一星评级）放在了五星评级应该在的地方，一些用户错误地为积极评价选择了它>

信号的可见性可以深刻影响用户行为、用户体验和反馈质量。用户在私下往往更加坦率——他们的活动被评判的可能性较低——这可能导致更高质量的信号。2024年，X（前身为Twitter）将“喜欢”设为私人。X的所有者Elon Musk声称，这一变更后喜欢的数量显著增加。

然而，私人信号可能降低可发现性和可解释性。例如，隐藏喜欢会阻止用户找到他们的连接已经喜欢的推文。如果X根据你关注的人的喜欢推荐推文，隐藏喜欢可能导致用户对为什么某些推文出现在他们的信息流中感到困惑。

反馈限制

用户反馈对应用开发者的价值毫无疑问。然而，反馈并非免费午餐。它有自己的限制。

偏见

像任何其他数据一样，用户反馈有偏见。了解这些偏见并围绕它们设计你的反馈系统很重要。每个应用都有自己的偏见。以下是一些反馈偏见的例子，让你了解需要注意什么：

宽容偏见

宽容偏见是人们倾向于比应得的更积极地评价项目的趋势，通常是为了避免冲突，因为他们感到

被迫要友好，或者因为这是最简单的选择。想象你匆忙中，一个应用要求你评价一次交易。你对交易不满意，但你知道如果你给它负面评价，你将被要求提供原因，所以你只是选择积极的，以便完成它。这也是为什么你不应该让人们为你的反馈做额外工作的原因。

在五星评级量表上，四星和五星通常表示良好体验。然而，在许多情况下，用户可能感到压力给予五星评价，保留四星用于出现问题的情况。根据Uber的说法，2015年，平均司机评级为4.8，评分低于4.6的司机面临被停用的风险。

这种偏见不一定是障碍。Uber的目标是区分好司机和坏司机。即使有这种偏见，他们的评级系统似乎帮助他们实现了这个目标。查看用户评级的分布以检测这种偏见至关重要。

如果你想要更细致的反馈，去除与低评分相关的强烈负面含义可以帮助人们摆脱这种偏见。例如，不是向用户显示数字一到五，而是向用户展示如下选项：

- "很棒的行程。很棒的司机。"
- "相当好。"
- "没什么可抱怨的，但也没什么特别的。"
- "本可以更好。"
- "不要再把我和这个司机匹配了。"

随机性

用户经常提供随机反馈，这不是出于恶意，而是因为他们缺乏提供更深思熟虑的输入的动力。例如，当并排显示两个长响应进行比较评估时，用户可能不想阅读两者，只是随机点击一个。在Midjourney的情况下，用户也可能随机选择一个图像来生成变体。

位置偏见

向用户呈现选项的位置会影响这个选项如何被感知。用户通常更可能点击第一个建议而不是第二个。如果用户点击第一个建议，这并不一定意味着它是一个好的建议。

设计反馈系统时，可以通过随机变化建议的位置或构建一个模型，根据建议的位置计算其真实成功率，来减轻这种偏见。

偏好偏见

许多其他偏见可能影响一个人的反馈，其中一些已在本书中讨论。例如，在并排比较中，人们可能更喜欢更长的响应，即使更长的响应不太准确——长度比不准确更容易注意到。另一个偏见是最近偏见，当比较两个答案时，人们倾向于青睐他们最后看到的答案。

检查你的用户反馈以揭示其偏见很重要。了解这些偏见将帮助你正确解释反馈，避免误导性的产品决策。

退化反馈循环

记住，用户反馈是不完整的。你只能获得关于你向用户展示的内容的反馈。

在使用用户反馈修改模型行为的系统中，可能出现退化反馈循环。当预测本身影响反馈，反馈又影响模型的下一次迭代时，可能发生退化反馈循环，放大初始偏见。

想象你正在构建一个推荐视频的系统。排名较高的视频首先出现，所以它们获得更多点击，强化系统认为它们是最佳选择的信念。最初，两个视频，A和B，之间的差异可能很小，但因为A排名略高，它获得了更多点击，系统不断提升它。随着时间的推移，A的排名飙升，让B落后。这个反馈循环是为什么受欢迎的视频保持受欢迎的原因，使新视频难以突破。这个问题被称为“曝光偏见”、“流行偏见”或“过滤气泡”，是一个被广泛研究的问题。

退化反馈循环可以改变你的产品的焦点和使用基础。想象一开始，少数用户给出反馈，表示他们喜欢猫照片。系统接收到这一点，并开始生成更多带有猫的照片。这吸引了猫爱好者，他们给出更多反馈，说猫照片很好，鼓励系统生成更多猫。不久，你的应用成为猫的天堂。在这里，我使用猫照片作为例子，但同样的机制可以放大其他偏见，如种族主义、性别歧视和对露骨内容的偏好。

根据用户反馈行动也可能将对话代理变成，缺乏更好的词，一个撒谎者。多项研究表明，在用户反馈上训练模型可以教它给用户它认为用户想要的东西，即使这不是最准确或最有益的(Stray, 2023)。Sharma等人(2023)表明，在人类反馈上训练的AI模型倾向于阿谀奉承。它们更可能呈现与用户观点匹配的用户响应。

用户反馈对改善用户体验至关重要，但如果不加区别地使用，它可能会延续偏见并破坏你的产品。在将反馈纳入你的产品之前，确保你了解这种反馈的限制及其潜在影响。

摘要

如果前面的每一章都专注于AI工程的特定方面，本章则整体看待了在基础模型上构建应用的过程。

本章分为两部分。第一部分讨论了AI应用的常见架构。虽然应用的确切架构可能各不相同，但这种高层次架构提供了一个框架，了解不同组件如何配合。我使用逐步方法构建这种架构，讨论每个步骤的挑战以及可以用来解决这些挑战的技术。

虽然有必要分离组件以保持系统模块化和可维护，但这种分离是流动的。组件在功能上可以有許多重叠方式。例如，护栏可以在推理服务中实现，也可以在模型网关中实现，或者作为独立组件实现。

每个额外的组件可能使你的系统更强大、更安全或更快，但也会增加系统的复杂性，使其面临新的失败模式。任何复杂系统的一个不可或缺的部分是监控和可观测性。可观测性涉及了解你的系统如何失败，围绕失败设计指标和警报，并确保你的系统设计使这些失败可检测和可追踪。

虽然许多软件工程和传统机器学习的可观测性最佳实践和工具适用于AI工程应用，但基础模型引入了新的失败模式，这需要额外的指标和设计考虑。

同时, 对话界面启用了新类型的用户反馈, 你可以利用它们进行分析、产品改进和数据飞轮。本章的第二部分讨论了各种形式的对话反馈, 以及如何设计你的应用以有效收集这些反馈。

传统上, 用户反馈设计被视为产品责任而非工程责任, 因此工程师通常会忽略它。然而, 由于用户反馈是持续改进AI模型的关键数据源, 现在更多的AI工程师开始参与这个过程, 以确保他们获得所需的数据。这强化了第1章的观点, 即与传统ML工程相比, AI工程正在向产品方向靠拢。这是因为数据飞轮和产品体验作为竞争优势的重要性都在增加。

许多AI挑战本质上是系统问题。要解决它们, 通常需要退后一步, 将系统作为一个整体考虑。单个问题可能由独立工作的不同组件解决, 或者解决方案可能需要多个组件的协作。彻底理解系统对于解决实际问题、开启新可能性和确保安全至关重要。